MLCN: MACHINE LEARNING CYBERSECURITY NETWORK

This project report is submitted to

Yeshwantrao Chavan College of Engineering

(An Autonomous Institution Affiliated to Rashtrasant Tukdoji Maharaj Nagpur University)

In partial fulfillment of the requirement For the award of the degree

Of

Bachelor of Technology in [BRANCH]

By

[STUDENT NAME(S) AND ROLL NUMBER(S)]

Under the guidance of

[GUIDE NAME]

DEPARTMENT OF [DEPARTMENT]

Nagar Yuwak Shikshan Sanstha's

YESHWANTRAO CHAVAN COLLEGE OF ENGINEERING,

(An autonomous institution affiliated to Rashtrasant Tukadoji Maharaj Nagpur University, Nagpur)

NAGPUR – 441 110

2025–26

CERTIFICATE OF APPROVAL

Certified that the project report entitled "MLCN: MACHINE LEARNING CYBERSECURITY NETWORK" has been successfully completed by [NAME OF THE STUDENT(S)] under the guidance of [NAME OF GUIDE] in recognition to the partial fulfillment for the award of the degree of Bachelor of Technology in [BRANCH], **Yeshwantrao Chavan College of Engineering *(An Autonomous Institution Affiliated to Rashtrasant Tukdoji Maharaj Nagpur University)***

Signature	Signature	Signature	Signature
[Name of Guide] (Guide)	[Name of Co-Guide] (Co-Guide)	[Name of Co-ordinator] (Coordinator)	[Name of Head of deptt] (HoD)

Unsigned approval template. Certification requires verification and signatures by the authorized faculty.

DECLARATION

I / we certify that

a. The work contained in this project has been done by me under the guidance of my supervisor(s).

b. The work has not been submitted to any other Institute for any degree or diploma.

c. I have followed the guidelines provided by the Institute in preparing the project report.

d. I have conformed to the norms and guidelines given in the Ethical Code of Conduct of the Institute.

e. Whenever I have used materials (data, theoretical analysis, figures, and text) from other sources, I have given due credit to them by citing them in the text of the report and giving their details in the references. Further, I have taken permission from the copyright owners of the sources, whenever necessary.

Signature of the Student/s: [TO BE SIGNED]

Name of the student	Signature of the Student	Name of the student	Signature of the Student
1. [NAME]	[SIGNATURE]	4. [NAME]	[SIGNATURE]
2. [NAME]	[SIGNATURE]	5. [NAME]	[SIGNATURE]
3. [NAME]	[SIGNATURE]	6. [NAME]	[SIGNATURE]

Unsigned declaration template for student verification. Unused student positions may be removed.

ACKNOWLEDGEMENT

The project team acknowledges the institutional framework provided by Yeshwantrao Chavan College of Engineering for the mini-project. This report documents the MLCN implementation and its technical evidence for review by the department and guide.

[STUDENT INPUT REQUIRED: identify the guide, co-guide, department, laboratory staff and any individuals who actually assisted the project. Describe the nature of their contribution and confirm permission to name them.]

The report recognizes the publicly documented contributions of the dataset providers and the research cited in the bibliography. The implementation uses open-source software including Scapy, XGBoost, NumPy and Streamlit. Original technical diagrams and evidence-based charts are prepared for this report.

[STUDENT INPUT REQUIRED: record any verified funding, external assistance or institutional disclosure required for document-generation support. No sponsorship or collaboration is asserted here.]

TABLE OF CONTENTS

Title Page..... i
CERTIFICATE OF APPROVAL..... ii
DECLARATION..... iii
ACKNOWLEDGEMENT..... iv
TABLE OF CONTENTS..... v
LIST OF TABLES..... vii
LIST OF FIGURES..... viii
LIST OF ABBREVIATIONS..... ix
LIST OF SYMBOLS / NOMENCLATURE..... x
ABSTRACT..... xi
PROJECT CO–PO MATRIX..... xii
Chapter 1 Introduction..... 1
1.1 Background and context..... 1
1.2 Network traffic and flow-based classification..... 2
1.3 Motivation and problem statement..... 2
1.4 Scope and boundaries..... 3
1.5 Objectives and contributions..... 4
1.6 Investigation method and evidence hierarchy..... 5
1.7 Organization of the report..... 5
Chapter 2 Review of Literature..... 6
2.1 Review strategy..... 6
2.2 Intrusion-detection approaches and operational context..... 6
2.3 Boosted trees and the implemented model choice..... 7
2.4 Traffic datasets and representation compatibility..... 8
2.5 Evaluation under imbalance and distribution shift..... 9
2.6 Explainability and narrative reporting..... 9
2.7 Critical comparison of the reviewed evidence..... 10
2.8 Related-patent search..... 11
2.9 Research gap and investigation direction..... 12
Chapter 3 Work Done / Present Investigation..... 13
3.1 Repository scope and evidence method..... 13
3.2 Overall architecture..... 13
3.3 Five-module processing pipeline and contracts..... 14
3.4 Module 1: packet capture..... 16
3.5 Module 2: packet parsing and normalization..... 16
3.6 Module 3: bidirectional flow construction..... 17
3.7 Module 4: canonical feature engineering..... 19
3.8 Module 5: XGBoost inference..... 22
3.9 Notebook training and evaluation workflow..... 24

3.10 Pipeline orchestration, completion and output....25
3.11 Traffic profiles, parameter mapping and generation....26
3.12 Receiver services and validation modes....28
3.13 Windows launcher and process lifecycle....28
3.14 Streamlit console and demonstration provenance....29
3.15 Test architecture and integration evidence....30
3.16 Proposed Gemini XAI architecture....30
Chapter 4 Results and Discussions....34
4.1 Evidence populations and experimental setup....34
4.2 Class distribution and imbalance....35
4.3 Overall offline classification results....36
4.4 Per-class results....37
4.5 Confusion matrix and error analysis....38
4.6 ROC and precision–recall analysis....40
4.7 Performance comparisons and baseline interpretation....42
4.8 Training provenance and notebook consistency....43
4.9 Resolution of the historical feature mismatch....43
4.10 Saved end-to-end laboratory validation....44
4.11 Executed tests and failure interpretation....45
4.12 Runtime observations and implementation costs....46
4.13 Threats to validity and supported interpretation....47
Chapter 5 Summary and Conclusions....48
5.1 Summary of the investigation....48
5.2 Objectives achieved....48
5.3 Conclusions derived from the results....49
5.4 Contributions and limitations....50
5.5 Specific future scope....50
Chapter 6 Appendix....52
6.1 Appendix I — Complete feature schema....52
6.2 Appendix II — Implementation inventory....53
6.3 Appendix III — Configuration and operating modes....55
6.4 Appendix IV — Reproducibility instructions....56
6.5 Appendix V — Proposed explanation contract....57
6.6 Appendix VI — Supplementary evidence and traceability....58
Chapter 7 References / Literature Cited....60
7.1 Bibliography in order of first citation....60
7.2 Related-patent verification scope....61
List of Publications....62

LIST OF TABLES

Table 1.1 Investigation objectives and evidence boundaries....4
Table 2.1 Critical literature comparison....10
Table 2.2 Verified related patent publications....12
Table 3.1 Core data contracts and interfaces....15
Table 3.2 Feature families and current semantics....20
Table 3.3 Recorded final notebook training configuration....25
Table 4.1 Result provenance and permitted interpretation....34
Table 4.2 Offline evaluation summary....36
Table 4.3 Verified per-class offline evaluation....37
Table 4.4 Exact confusion matrix (actual rows, predicted columns)....39
Table 4.5 Regenerated one-versus-rest ranking metrics....42
Table 4.6 Same-test-population baseline comparison....42
Table 4.7 Saved laboratory validation, opening-flow denominator....44
Table 4.8 Failures in the 202-test audit run....46
Table 5.1 Objective achievement and remaining work....48
Table 6.1 Complete ordered 24-feature runtime contract....52
Table 6.2 Audited implementation and artifact inventory....54
Table 6.3 Configuration by execution context....55
Table 6.4 Reproducibility and laboratory command reference....56
Table 6.5 Proposed Gemini evidence and response contract....57
Table 6.6 Local CIC CSV row counts....58
Table 6.7 Selected artifact SHA-256 hashes....59
Table P.1 Candidate publications status....62

LIST OF FIGURES

Fig. 3.1 Overall MLCN architecture. Solid blocks show audited components; the dashed Gemini block is proposed. This is an architectural diagram, not a measured network topology. 14

Fig. 3.2 Five-module processing pipeline and typed handoffs. The pipeline orchestrator connects existing module APIs and retains the genuine combined result.....15

Fig. 3.3 Packet-to-flow transformation, completion and classification eligibility. Discard and completion rules are architectural conditions, not experimental detection rates.....17

Fig. 3.4 Bidirectional key construction and orientation. Canonical endpoint ordering and forward-direction anchoring serve different purposes.....18

Fig. 3.5 Feature-engineering workflow under the current 24-feature contract. Timing features use microseconds; rates use seconds.....20

Fig. 3.6 XGBoost loading and inference. Confidence is the maximum model probability and is not a calibrated probability of a real intrusion.....23

Fig. 3.7 Recorded training/evaluation methodology and audit replay. Early notebook outputs are inconsistent; the saved test artifacts and regenerated inference are independently checked. 24

Fig. 3.8 Generator/receiver interaction and evidence paths. Profile reference medians do not bypass capture or enter the detector as synthetic feature vectors.....27

Fig. 3.9 Proposed Gemini explanation architecture. Dashed blocks are unimplemented design elements; only genuine evidence is eligible for explanation.....31

Fig. 4.1 Verified class distribution in models/xgboost_test_predictions.csv, n = 251,812. Bar heights use a logarithmic count axis.....35

Fig. 4.2 Per-class precision, recall and F1 reconstructed from saved offline labels. Brute Force precision is substantially below the other classes.....38

Fig. 4.3 Verified offline confusion matrix. Cell annotations are exact counts; colour represents log10(count + 1). Rows are actual and columns predicted.....39

Fig. 4.4 One-versus-rest ROC curves regenerated from the saved booster and test features using XGBoost 2.1.4. These are offline ranking results.....41

Fig. 4.5 One-versus-rest PR curves from regenerated probabilities. The black point shows the separate Brute Force multiclass argmax operating point.....41

Fig. 4.6 Saved laboratory opening-flow counts, reconciled using consecutive JSONL blocks and the saved report’s profile totals. Matching is relative to the selected profile.....45

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AP	Average precision
API	Application programming interface
BPF	Berkeley Packet Filter
CIC	Canadian Institute for Cybersecurity
CLI	Command-line interface
CO / PO	Course outcome / program outcome
CSV	Comma-separated values
DDoS / DoS	Distributed denial of service / denial of service
FIN / SYN / RST	Finish / synchronize / reset TCP control flags
FP / FN / TP	False positive / false negative / true positive
HTTP	Hypertext Transfer Protocol
IAT	Inter-arrival time
ICMP	Internet Control Message Protocol
IDS	Intrusion detection system
IP	Internet Protocol
JSON / JSONL	JavaScript Object Notation / JSON Lines
LIME	Local Interpretable Model-agnostic Explanations
LLM	Large language model
MLCN	Machine Learning Cybersecurity Network
PR / ROC	Precision–recall / receiver operating characteristic
ROC-AUC	Area under the receiver operating characteristic curve
SHA-256	Secure Hash Algorithm, 256-bit digest
SHAP	SHapley Additive exPlanations
TCP / UDP	Transmission Control Protocol / User Datagram Protocol
UTC	Coordinated Universal Time
XAI	Explainable artificial intelligence

LIST OF SYMBOLS / NOMENCLATURE

Symbol	Meaning / unit
D	Flow duration, microseconds
g_i	Consecutive inter-arrival gap, microseconds
m	Number of values in a statistical series
n	Total packets in a completed flow
n_b	Backward packet count
n_f	Forward packet count
p_k	Model probability for class k
s	Sample standard deviation in the stated feature unit
t_i	Timestamp of packet i, integer microseconds
x_j	Value of model feature j
μ	Arithmetic mean
μs	Microseconds (10^{-6} seconds)

Symbols are arranged by Latin letters followed by Greek letters. Subscripts indicate packet, feature, class or direction indices. Named code objects and literal feature names retain their repository spelling.

ABSTRACT

Network intrusion detection requires a consistent transformation from observed packets to the numerical representation used by a trained classifier. This project investigates Machine Learning Cybersecurity Network (MLCN), a laboratory platform connecting packet capture, parsing, bidirectional flow construction, 24-feature extraction and five-class XGBoost inference. The current repository, training notebook, model artifacts and validation logs are audited, and all 251,812 saved test predictions are independently replayed. Offline accuracy is 99.4238%, balanced accuracy is 99.5568%, and macro F1 is 0.93358. Brute Force precision remains limited to 53.1378%, with 799 false positives. Current code corrects the historical timing-unit mismatch by using microsecond timing features and per-second rates. Saved laboratory records provide narrower evidence of selected-profile recognition and are separated from synthetic demonstration output. Six of 202 executed tests fail in demonstration labelling or presentation behavior. A Gemini-based explanation workflow is proposed with explicit evidence, privacy and uncertainty controls; it is not implemented. The findings support a connected research prototype while identifying reproducibility, provenance and live-validation requirements before broader deployment.

PROJECT CO–PO MATRIX

Course outcome (CO) and program outcome (PO) mappings require approval from the department. The placeholders below do not assert any achieved mapping or correlation level. Replace the row labels, number of outcomes and every pending cell with the institution-approved values.

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
[CO 1]	—	—	—	—	—	—	—	—	—	—	—	—
[CO 2]	—	—	—	—	—	—	—	—	—	—	—	—
[CO 3]	—	—	—	—	—	—	—	—	—	—	—	—
[CO 4]	—	—	—	—	—	—	—	—	—	—	—	—

Legend: — = mapping pending, not zero and not “no correlation”. No numerical strength scale is assigned without the approved institutional rubric.

[REQUIRED INPUT: official course outcomes, program outcomes, approved matrix, correlation scale, evaluator and approval date.]

Suggested evidence for departmental review includes modular implementation, experimental analysis, reproducibility, technical documentation and responsible interpretation of model output. These are project activities, not approved CO–PO assignments.

Chapter 1
Introduction

1.1 Background and context

Machine Learning Cybersecurity Network (MLCN) is a laboratory implementation of flow-based network intrusion detection. An intrusion detection system (IDS) observes activity and identifies patterns that warrant investigation. A network IDS derives these observations from communication traffic. It differs from a preventive control because a classification does not itself block a connection or establish that a compromise occurred. MLCN connects packet capture, parsing, conversation assembly, numerical feature extraction and supervised classification in one executable processing chain.

The practical difficulty is maintaining agreement between the data representation used during model development and the representation produced by a running network monitor. A high score on previously extracted rows cannot demonstrate that live packets are grouped, measured and classified correctly. Sommer and Paxson [1] identify the gap between controlled machine-learning evaluations and operational intrusion detection. In MLCN, this concern becomes concrete through timing units, payload accounting, flow termination and the distinction between genuine model predictions and demonstration output.

The present investigation examines the repository at commit 95407efc1f5d2b413abc36b24824481df97f7306, together with the local files available during the audit of 21 October 2026. The supplied implementation guide provides orientation, but current executable code and saved artifacts determine the technical findings. The report is an assessment of the observed implementation and its evidence, rather than a claim that every feature mentioned in the root README has been completed. The report follows the supplied institutional academic-year template, 2025 –26; the actual submission date remains subject to student confirmation.

1.2 Network traffic and flow-based classification

Individual packets carry addresses, protocol identifiers, flags and payload. Many useful indicators depend on relationships among packets: the balance of requests and responses, elapsed time, inter-arrival gaps and characteristic message lengths. A bidirectional flow groups packets belonging to the same pair of transport endpoints and protocol. Aggregating such packets provides a compact representation for a classifier, while retaining enough temporal and directional structure for several common traffic categories.

The Canadian Institute for Cybersecurity (CIC) publishes CIC-IDS2017 as an intrusion-detection dataset with packet captures and labelled flow records [2]. The local MLCN workspace contains eight MachineLearningCVE comma-separated-value (CSV) files associated with that dataset and a notebook that selects five output classes. The measured unit of evaluation is a row representing a flow. The 2,830,743 rows found across these files must therefore not be described as an equivalent number of captured packets.

The implemented classifier predicts BENIGN, Brute Force, distributed denial of service (DDoS), denial of service (DoS), or Port Scan. BENIGN is the non-attack class, so the system contains four attack labels and one normal label. These classes do not cover all hostile activity. The notebook excludes other original categories, including Bot, Web Attack, Infiltration and Heartbleed, from the final five-class task. A prediction is consequently a choice within a restricted label vocabulary, not an open-ended assessment of all network threats.

1.3 Motivation and problem statement

The project is motivated by the need to make a trained network classifier observable as a complete system. A classifier deployed without documented packet semantics can receive plausible-looking but incompatible inputs. Conversely, a functioning capture interface can produce no useful predictions if flows never reach the completion boundary or if traffic falls outside the model’s classified protocols. Both types of failure can be hidden by a user interface that displays a desired profile instead of the actual detector output.

The problem addressed is to implement and assess an auditable conversion from observed network packets to a fixed 24-feature vector and a five-class prediction. The associated research questions are whether the repository preserves a consistent feature contract; whether its saved offline results can be independently reproduced; what the available laboratory records establish about live operation; and what additional evidence is necessary before an explanation service can responsibly describe a detection.

Extreme Gradient Boosting (XGBoost), developed by Chen and Guestrin [3], supplies the tree-ensemble classifier. Its use is technically appropriate for structured numerical inputs, but the algorithm's general success does not establish performance on an unseen network. The report therefore treats algorithm selection, implementation correctness, statistical evaluation and deployment validity as separate questions. This separation also prevents a demonstration profile match from being mistaken for independently labelled attack detection.

1.4 Scope and boundaries

The verified scope includes five connected processing modules, a trained XGBoost artifact, a training/evaluation notebook, local CIC flow data, a bounded traffic generator, receiver services, a Windows launcher, a Streamlit operations console, validation utilities and tests. The pipeline retains genuine results in memory and can emit structured detection records through its command-line interface (CLI). A separate demonstration transformation can alter displayed predictions when explicitly enabled or enabled by launcher defaults; its presence is central to interpreting the evidence.

The project does not verify network-wide visibility, high-volume capture reliability, autonomous remediation, authentication of an attacker, identification of malware families or generalization to arbitrary environments. The implemented feature vector contains no direct application credential content. The Brute Force generator produces labelled laboratory request/deny exchanges rather than guessing real credentials. The DDoS-labelled profile is bounded local traffic from the generator, so it is not evidence of a distributed attack campaign or a service outage.

Explainable artificial intelligence (XAI) using Gemini is included as a proposed extension. No implemented Gemini request path, model-attribution service or incident-report generation module is established by the source audit. A proposal is developed in Chapter 3 with explicit contracts and acceptance criteria. The application source is preserved during this reporting investigation; measured defects and limitations are documented instead of being silently repaired.

1.5 Objectives and contributions

The investigation has six objectives: trace the complete executable processing path; document the exact feature semantics; establish provenance for model-development and evaluation artifacts; reproduce available offline metrics; distinguish saved live validation from simulations and fabricated display values; and specify a grounded explanation workflow. Each objective is evaluated through code inspection, artifact checks or an explicitly identified design analysis.

The implementation contribution is the integration of packet capture with a schema-constrained five-class detector and a controlled laboratory demonstration environment. The reporting contribution is an evidence-based account of that system. It includes corrected interpretation of the confusion matrix, confirmation that the previously documented timing mismatch is repaired in current code, recognition of notebook execution inconsistencies, and identification of demonstration-source labelling failures in the test suite.

Table 1.1 maps the objectives to evidence and appropriate completion claims. A distinction is maintained between a capability implemented in code and a capability validated under representative deployment conditions. This distinction permits useful conclusions about engineering integration without extending the offline accuracy claim to unmeasured operational settings.

Table 1.1 Investigation objectives and evidence boundaries

Objective	Evidence	Permitted claim
Trace integration	Source modules and handoff tests	Implemented packet-to-prediction chain
Specify features	contract.py, engine.py and schema	Current local semantics documented

Objective	Evidence	Permitted claim
Verify offline results	Saved test CSV, model and metrics	Replayed labels and recomputed scores
Assess laboratory operation	Saved logs and simulation tests	Recorded selected-profile behavior
Audit presentation integrity	Fallback code, JSONL and tests	Synthetic output identified
Design Gemini extension	Proposed contracts and literature	Design only; no API implementation

1.6 Investigation method and evidence hierarchy

The institutional PDF determines formatting and report organization. Repository code and machine-readable artifacts determine technical claims, followed by the implementation guide where it remains consistent. External literature supplies methodological context and independently verifiable related work. The audit records file hashes, model metadata, local dataset row counts, complete saved evaluation predictions and the results of the repository test suite. Generated plots use these artifacts or regenerated probabilities from the saved model.

Three evidence levels are used throughout. Recomputed findings arise from execution during this audit, including all saved-label metrics and regenerated probability curves. Recorded findings arise from existing notebook outputs or laboratory logs whose original collection process is not rerun. Proposed elements describe future behavior and contain no measured performance claim. An architectural figure explains software relationships; an experimental figure summarizes a specified population of observations.

1.7 Organization of the report

Chapter 2 reviews literature and patents. Chapter 3 documents the implementation and proposed Gemini design. Chapter 4 analyses offline results, laboratory records and test failures. Chapter 5 presents conclusions and future work. Chapter 6 supplies technical appendices, and Chapter 7 lists references, followed by the publications table.

Chapter 2
Review of Literature

2.1 Review strategy

The review concentrates on work that helps assess the implemented task: flow-based supervised classification, tree ensembles, traffic-data quality, imbalanced evaluation and explanations. Sources are checked through original papers, author-maintained research pages, dataset-provider websites and patent publication records. Search terms include “machine learning network intrusion detection”, “flow classification”, “CICIDS2017 feature extraction”, “precision recall imbalanced datasets” and “adaptive network intrusion detection patent”. Verification is performed on 21 October 2026.

The review is selective rather than exhaustive. It does not combine accuracy figures from incompatible datasets into an apparent league table. Differences in labels, class balance, sampling, preprocessing and train/test separation can dominate an algorithm comparison. The comparisons below instead identify methodological choices and their relevance to MLCN. Numerical comparisons within this report are restricted to the same saved evaluation population or are clearly labelled as results from a different evidence level.

2.2 Intrusion-detection approaches and operational context

Signature-based detection searches for predefined evidence of known activity, whereas supervised classification learns relationships between labelled examples and outputs. Anomaly detection estimates a reference distribution and flags deviations. These approaches answer different questions: deviation from a baseline need not be malicious, and membership in a learned attack class does not establish the precise cause of an event. MLCN implements supervised multiclass classification, even though parts of its motivation use the broader language of anomaly detection.

Sommer and Paxson [1] emphasize the difficulty of transferring machine learning into open operational environments. Their discussion motivates a requirement for realistic

benign traffic and an understanding of the cost of errors. The present report applies that requirement by examining Brute Force precision separately from aggregate accuracy and by refusing to treat generated laboratory labels as independently established truth.

Handley, Paxson and Kreibich [4] study ambiguity between traffic observed by a network monitor and traffic interpreted by endpoints. Their traffic-normalization work is conceptually different from MLCN's statistical classifier, but it highlights why capture semantics matter. The local pipeline parses packets independently and does not reconstruct complete application sessions. Its predictions therefore inherit uncertainty from missing packets, segmentation, reordering and visibility across interfaces.

2.3 Boosted trees and the implemented model choice

Chen and Guestrin [3] present XGBoost as a scalable tree-boosting system with regularization and engineering techniques for efficient learning. For MLCN, the relevant property is the ability to model nonlinear relationships among numerical flow features without adding a learned scaling stage. The repository's final training cell uses a multiclass probability objective and histogram-based tree construction. These are code-verified choices, not deductions from the article.

A single decision tree offers a small rule structure but may be sensitive to training variation. A random forest averages multiple trees, while boosting constructs an additive ensemble whose later steps reduce the objective remaining after earlier steps. A neural network can represent complex interactions but introduces additional architectural and optimization choices. These are methodological alternatives; the repository does not supply matched final test artifacts for them. MLCN's saved XGBoost results cannot establish superiority over these alternatives without controlled experiments.

The key empirical question is not whether boosted trees are powerful in general, but whether the chosen representation remains stable from training to live operation. A tree threshold learned in microseconds will not retain the same meaning if seconds are

supplied. Likewise, a flag column interpreted as a packet count differs fundamentally from a dataset-specific first-packet binary encoding. The audited feature contract addresses these issues directly and is more consequential to this integration study than an unsupported comparison of classifier brands.

2.4 Traffic datasets and representation compatibility

CIC-IDS2017 provides a relevant source because its labelled flows and attack categories overlap the implemented classifier. The provider describes a five-day collection period, 3–7 July 2017, and traffic processed through CICFlowMeter [2]. The notebook's introductory statement about seven days is therefore not retained as a verified dataset fact. Local CSV names and raw row counts corroborate the presence of the eight daily/attack-specific files; they do not independently certify the origin of every row against a provider checksum.

CIC-DDoS2019 broadens the discussion to multiple DDoS families and separately described collection periods [5]. It is useful as a future external-domain evaluation source, but its inclusion in documentation does not prove that it trained the saved MLCN model. Matching feature names across datasets does not establish matching extractor versions, label definitions or sampling conditions. The present model is therefore described in relation to the locally evidenced CIC-IDS2017 workflow.

The Stratosphere Laboratory's CTU-13 dataset contains thirteen botnet scenarios and distinguishes botnet, normal and background traffic [6]. Its bidirectional Argus flow representation illustrates that "flow-based" does not mean schema-identical. MLCN's five labels exclude a dedicated botnet class, and its 24-column CIC compatibility contract is not an Argus adapter. A credible cross-dataset study would need an explicit label harmonization and feature reconstruction procedure before evaluating transfer.

Engelen, Rimmer and Joosen [7] identify problems in CICIDS2017 traffic generation, flow construction, extraction and labelling. Their study supports scrutiny of the measurement pipeline itself. For MLCN, reproducing historical extractor behavior may improve compatibility with a saved model while retaining peculiarities that should not be mistaken for universal networking definitions. The documented binary

flag permutation and duplicated first payload in aggregate length statistics are reported as local compatibility rules, not generalized definitions of those feature names.

2.5 Evaluation under imbalance and distribution shift

Arp, Quiring, Pendlebury, Warnecke, Pierazzi, Wressnegger, Cavallaro and Rieck [8] review common pitfalls in machine-learning security research. Their recommendations motivate checking data provenance, evaluation design and the connection between a measured task and a security claim. The present audit finds a stratified row split in the notebook, along with preprocessing performed before splitting. This is stronger evidence than an unknown split procedure, but it leaves dependence among related flows and preprocessing leakage as concrete concerns.

Saito and Rehmsmeier [9] explain why precision–recall views are valuable for imbalanced classification. MLCN's Brute Force class is only 915 of 251,812 test rows. Its high recall is compatible with low precision because a small fraction of the much larger benign population can create many false alerts. Reporting both receiver operating characteristic (ROC) and precision–recall (PR) curves helps expose the difference between score ranking and a selected decision rule.

An area under the ROC curve (ROC-AUC) summarizes ranking across thresholds; it is not a statement that every deployed alert has that probability of being correct. The current multiclass decision rule selects the greatest probability without a separate rejection threshold. Its precision is evaluated at that specific operating point. Probability calibration, threshold selection and out-of-distribution rejection would require their own validation protocol, preferably using a population representative of the intended deployment.

2.6 Explainability and narrative reporting

Lundberg and Lee [10] introduce SHapley Additive exPlanations (SHAP), an additive feature-attribution framework. Ribeiro, Singh and Guestrin [11] develop Local Interpretable Model-agnostic Explanations (LIME), based on local surrogate explanations. These methods seek to relate a model output to input features under their

respective assumptions. Neither establishes that a suspicious network event caused an intrusion, and neither is verified as an implemented MLCN component.

A large language model (LLM) can translate supplied evidence into a readable incident narrative. That ability is distinct from calculating an attribution. A Gemini response should not invent a statement that “high SYN count caused the prediction” when no attribution was computed and the named column follows a nonstandard flag mapping. The proposed design therefore uses a factual evidence contract and reports attribution as unavailable unless a separate, validated method has produced it.

Google’s Gemini documentation describes schema-constrained structured output using JavaScript Object Notation (JSON) [12]. Syntactic conformity is useful for reliable integration, but it cannot establish factual correctness. The proposal in Chapter 3 adds evidence identifiers, numerical consistency checks, explicit uncertainty and a deterministic fallback. This is a system-design recommendation for MLCN; no application programming interface (API) execution or explanation-quality result is claimed.

2.7 Critical comparison of the reviewed evidence

Table 2.1 summarizes the relationship between the reviewed work and the implemented project. The comparison concerns methodological relevance rather than copying published performance scores. It identifies where the current system has an implementation response and where empirical work remains necessary.

Table 2.1 Critical literature comparison

Work / source	Relevant contribution	MLCN implication / limitation
Sommer and Paxson [1]	Operational gap for ML intrusion detection	Offline evaluation requires live-domain evidence.
Chen and Guestrin [3]	Scalable regularized tree boosting	Supports model rationale; no project-specific guarantee.
CIC-IDS2017 [2]	Labelled network-flow benchmark	Local training evidence; restricted five-class task.
CIC-DDoS2019 [5]	Broader DDoS-family data	Future transfer study; not verified training input.

Work / source	Relevant contribution	MLCN implication / limitation
CTU-13 [6]	Botnet scenarios and bidirectional flows	Different labels and extractor representation.
Engelen et al. [7]	Dataset and extractor quality analysis	Compatibility does not imply semantic correctness.
Arp et al. [8]	Security-ML evaluation pitfalls	Persist lineage and avoid split leakage.
Saito and Rehmsmeier [9]	PR interpretation under imbalance	Inspect rare-class precision and thresholds.
Lundberg and Lee [10]; Ribeiro et al. [11]	Feature-attribution and local surrogate methods	Separate attributions from LLM narrative.

The literature supports treating an IDS as a measurement-and-decision system. In MLCN, packet parsing determines the observations, flow construction determines the unit of analysis, feature extraction determines the statistical meaning of inputs, and inference determines a restricted label. A dashboard or narrative service is downstream of these stages and cannot correct missing evidence simply by presenting a more confident explanation.

2.8 Related-patent search

An accessible patent search is performed through Google Patents using combinations of network intrusion detection, machine learning, flow features, real-time classification and adaptive representation. Two publications are selected because their claims and technical descriptions are directly relevant to the project. Publication identifiers, titles, inventors, dates and the cited technical concepts are checked on the publication pages. This is a bounded related-work search; it is not a legal opinion on novelty, validity or freedom to operate.

Powell, Drozdenko and Roodbeen’s US12143406B1 [13], published 12 November 2024, describes real-time network intrusion detection using statistical measures and machine learning. Its discussion of similarity between training measurements and monitored traffic is relevant to MLCN’s feature-contract problem. MLCN uses a particular five-class XGBoost model and explicitly documented extraction semantics; those implementation choices alone do not establish a patentable distinction.

Papamartzivanos, Bifulco, Kambourakis and Gomez Marmol's US20220021695A1 [14], published 20 January 2022, describes adaptive intrusion detection through a learned representation of unlabelled traffic and modification of labelled training data. MLCN currently applies a fixed saved model and does not perform that adaptive representation-learning procedure. The comparison identifies adaptation to network changes as future research rather than a completed capability.

Table 2.2 records the verified publications and their limited relevance. Patent legal-status labels are not used as technical evidence, and no assertion of project patentability follows from the search. Further searches across patent families, claims and jurisdictions would be needed if a formal intellectual-property assessment were requested.

Table 2.2 Verified related patent publications

Publication	Verified dates / inventors	Relevant distinction
US12143406B1 [13]	Priority: 1 Sep 2022; published: 12 Nov 2024. Powell, Drozdenko, Roodbeen.	Statistical compatibility between training and monitored traffic; MLCN documents a fixed feature contract.
US20220021695A1 [14]	Priority: 9 Nov 2018; published: 20 Jan 2022. Papamartzivanos, Bifulco, Kambourakis, Gomez Marmol.	Adaptive representation and training-set modification; MLCN has no such adaptation.

2.9 Research gap and investigation direction

The gap addressed by this project is the distance between an offline flow classifier and an inspectable packet-to-prediction implementation. The reviewed sources motivate measuring that distance through explicit semantics and realistic evaluation. The repository provides a useful integration case because it contains both strong offline results and laboratory mechanisms whose outputs have different evidentiary meanings.

The investigation therefore traces implemented handoffs, checks feature semantics, reproduces saved results and separates genuine laboratory evidence from display substitutions. It proposes an evidence-grounded explanation layer. Independent live-labelled captures, session-aware evaluation, improved Brute Force precision and reliable lifecycle handling remain the principal research needs.

Chapter 3
Work Done / Present Investigation

3.1 Repository scope and evidence method

The repository audit covers application packages, configuration, models, data, images, tests, documentation, notebook contents and logs. The generated inventory records 133 substantive files after excluding version-control internals, virtual environments, operating-system metadata, bytecode caches and newly generated report files. Each included file is read and hashed; Python modules are additionally parsed for class and function definitions. The inventory is retained with the report-generation evidence, while selected interfaces are documented in Appendix II.

The most important artifacts are the saved model, its ordered feature/class metadata, the complete saved test prediction table, the evaluation summary, the confusion matrix and the training notebook. Source hashes associate the report with an exact local state. They do not establish who created an artifact or validate an external dataset download. The institutional PDF and implementation guide are also inventoried, with current code taking precedence wherever the guide has become outdated.

Existing image assets are inspected as project documentation but are not reused as technical evidence. The diagrams in this report are generated originally from the audited implementation. A Gemini-generated illustration in the assets directory does not establish an implemented Gemini integration. Similarly, the root README's proposed explanation, incident database and reporting modules are separated from the implemented capture, classification and operations-console functionality.

3.2 Overall architecture

Fig. 3.1 shows the complete implemented architecture and the proposed narrative extension. The generator sends controlled traffic to a receiver; capture observes packets rather than receiving reference medians from the profile file. The detector consumes features derived from completed flows. Genuine classification results are

preserved before any optional display transformation. The Streamlit console reads persisted receiver records and displays the current run and recent flow history.

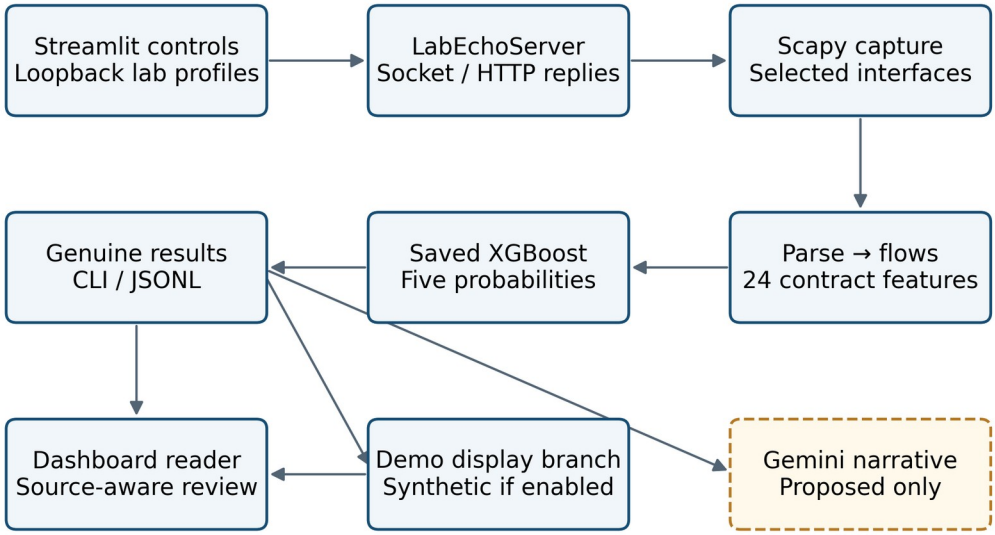


Fig. 3.1 Overall MLCN architecture. Solid blocks show audited components; the dashed Gemini block is proposed. This is an architectural diagram, not a measured network topology.

The architecture has two important separations. First, traffic generation produces an experimental stimulus, while packet capture independently observes the resulting traffic. Second, the genuine prediction and the displayed prediction may differ when demonstration fallback is enabled. Scientific evaluation must follow the genuine result through both separations. A profile name or an attractive display card cannot replace either captured evidence or a classifier output.

3.3 Five-module processing pipeline and contracts

Fig. 3.2 expands the five connected modules. Module 1 captures raw Scapy packets. Module 2 produces PacketMetadata. Module 3 groups metadata into Flow objects. Module 4 computes an ordered FeatureVector. Module 5 produces a DetectionResult, and orchestration combines the flow, features and prediction in PipelineResult. These typed objects make responsibilities explicit and allow tests to enter the system at different boundaries.

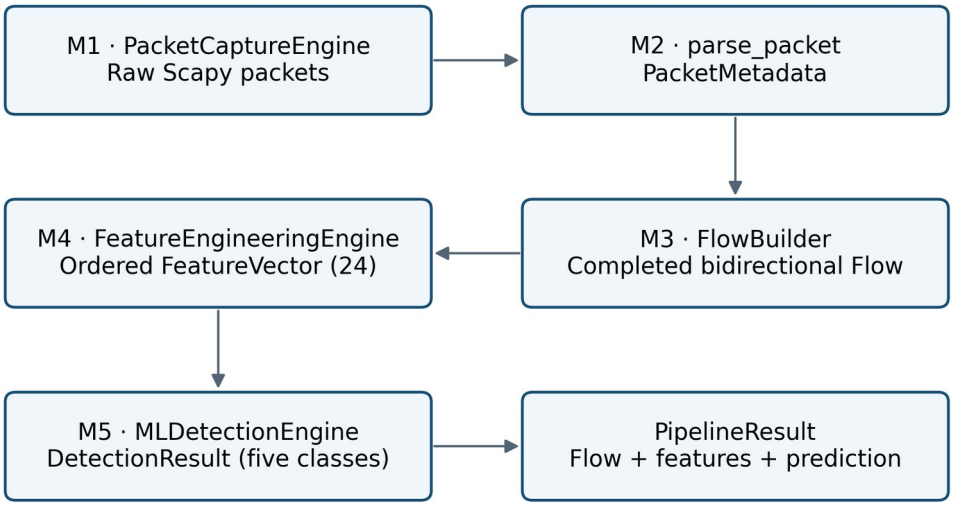


Fig. 3.2 Five-module processing pipeline and typed handoffs. The pipeline orchestrator connects existing module APIs and retains the genuine combined result.

Table 3.1 describes the main interfaces. Intermediate objects are immutable dataclasses where defined, but the active-flow accumulator is mutable. A completed flow retains all packet metadata, enabling feature recomputation without parsing the original Scapy packet again. This improves inspectability but also means memory use grows with retained packet histories and accumulated results, not merely with the number of active flow keys.

Table 3.1 Core data contracts and interfaces

Boundary	Input / interface	Output / constraint
Capture → parse	capture_metadata(callback), raw Scapy Packet	PacketMetadata or skipped parse
Parse → flow	process_metadata; FlowBuilder.add_packet	Canonical key, retained packet history
Flow → features	FeatureEngineeringEngine.extract(Flow)	24 values, exact order and units
Features → model	MLDetectionEngine.predict(FeatureVector)	Five probabilities and argmax class
Model → output	PipelineResult; CLI callback	Genuine internal result; optional transformed display
Output → console	JSONL read_records / build_view	Latest flow or session verdict; bounded file tail

3.4 Module 1: packet capture

PacketCaptureEngine in packet_capture/capture.py resolves an interface or a comma-separated interface list and invokes Scapy sniff with store=False and a one-second timeout. A Berkeley Packet Filter (BPF) expression can restrict traffic to a laboratory port or port range. The callback path capture_metadata is the integration interface used by the pipeline. Separate display-oriented methods preserve the packet-capture CLI and its ParsedPacket representation.

Single-interface capture runs directly; multiple interfaces create daemon worker threads. The implementation installs signal handlers only from the main thread, allowing background validation to start without an illegal signal registration. On Windows, Ctrl+Break support helps the launcher stop a child process group. Permissions and interface errors are surfaced through user-facing CLI messages. In contrast, callback exceptions are generally logged at debug level and suppressed so that capture can continue.

The use of store=False prevents Scapy from retaining its own full packet list, but it does not make the whole application constant-memory. The flow builder stores metadata and the pipeline retains completed results. Multiple interface workers can also call into shared flow state without an explicit pipeline lock. This is a code-level concurrency concern, not a measured corruption result. No packet deduplication, capture-loss measurement or global ordering queue is verified.

3.5 Module 2: packet parsing and normalization

The packet parser accepts a Scapy Packet and returns PacketMetadata or None. It normalizes timestamps to timezone-aware Coordinated Universal Time (UTC), validates Internet Protocol (IP) addresses, bounds port numbers, extracts Transmission Control Protocol (TCP) or User Datagram Protocol (UDP) fields, and records packet length. IPv4 time to live and IPv6 hop limit are exposed as metadata, together with TCP window information when applicable. These auxiliary fields are not all included in the 24-feature model vector.

Non-IP frames can produce metadata with absent addresses; such records are later unable to form a valid flow key. Malformed objects, unusable lengths and invalid addressing are skipped. TCP flags are normalized into a compact letter representation. The parser recognizes Internet Control Message Protocol (ICMP) for metadata purposes, but the integrated classifier subsequently excludes protocols other than TCP and UDP. Parsing support must therefore not be confused with classification coverage.

Fig. 3.3 shows the transformation from packets into eligible flows. The parser distinguishes observed packet length from transport payload length. Payload accounting subtracts Scapy padding already represented in the transport payload and applies an IPv4 minimum-frame padding adjustment. This is a compatibility rule intended to align short-packet lengths with the historical CIC-derived representation. IPv6 does not follow that IPv4 padding branch. The report does not claim independent packet-capture equivalence for every protocol or platform.

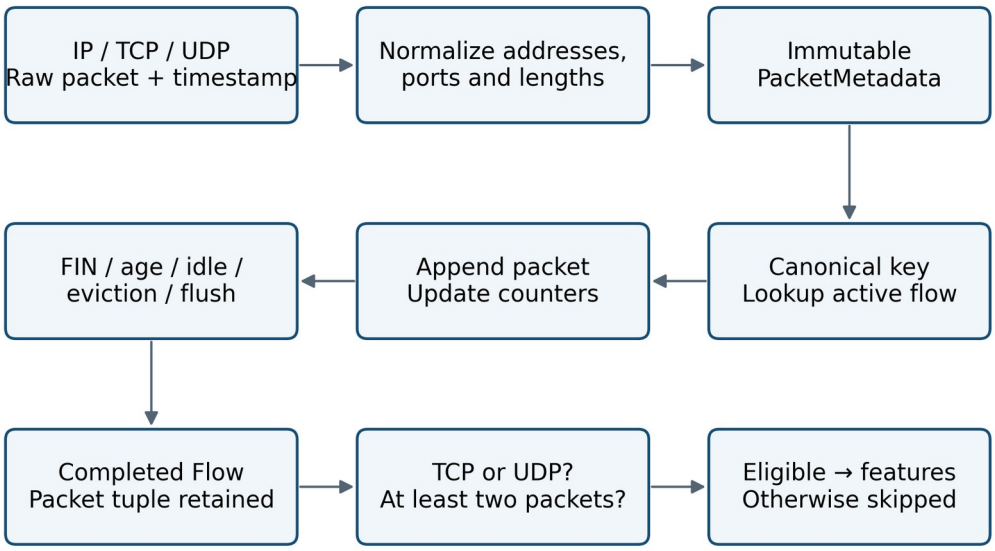


Fig. 3.3 Packet-to-flow transformation, completion and classification eligibility. Discard and completion rules are architectural conditions, not experimental detection rates.

3.6 Module 3: bidirectional flow construction

A FlowKey is constructed from source address, destination address, source port, destination port and normalized protocol. The endpoint pairs are sorted lexicographically, using a sentinel only when a port is absent. A packet and its reverse

direction therefore map to the same dictionary key. Canonical endpoint ordering is a lookup convention; it is not a declaration that the smaller address is the client or that the key's source endpoint initiated the connection.

The mutable _ActiveFlow stores the first arrival's direction as its forward orientation and appends subsequent metadata. It tracks packet counts, observed byte counts, direction-specific counts and timestamps. Completion creates an immutable Flow with the packet tuple and a closed_by_fin flag where appropriate. Fig. 3.4 separates the identity of the conversation from its direction convention. This distinction is necessary when explaining directional features or interpreting logs whose canonical source is the receiver port.

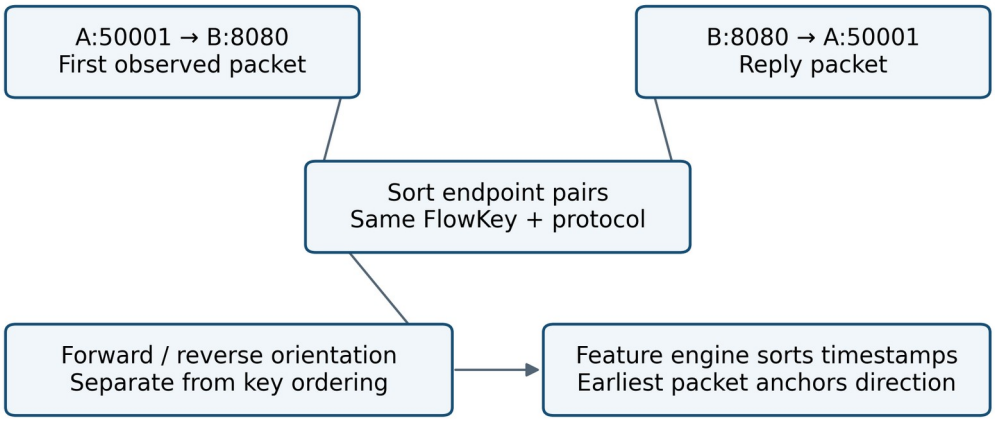


Fig. 3.4 Bidirectional key construction and orientation. Canonical endpoint ordering and forward-direction anchoring serve different purposes.

The generic FlowBuilder defaults are a 60-second inactivity timeout, a 300-second maximum duration, 100,000 active flows and FIN termination disabled. The integrated pipeline overrides the maximum duration to 120 seconds and enables FIN termination through the feature contract. The launcher separately shortens inactivity to five seconds for laboratory runs. A report that quotes only the standalone class defaults would misdescribe the actual integrated system.

On each accepted packet, the builder first expires flows that have exceeded the inactivity limit relative to that packet's timestamp. For an existing key, it also checks maximum age before appending. A FIN packet appended to an existing flow can close

it immediately; a first packet starting a new flow is not subjected to the same post-append FIN branch. Capacity eviction selects the least recently seen active flow, and an explicit flush closes every remaining flow. TCP reset alone is not a configured termination condition.

The expiration scan traverses active flows, so add_packet is not constant-time in the number of active conversations. During a completely silent capture, no background timer in the builder independently expires idle flows. A later packet or explicit shutdown flush triggers completion. This explains why an idle timeout is not itself a guaranteed maximum alert delay. The same code also updates end_time in arrival order, whereas feature extraction later sorts packet timestamps; out-of-order arrivals can therefore create differences between displayed duration and the model's duration feature.

3.7 Module 4: canonical feature engineering

The canonical contract is defined in feature_engineering/contract.py and re-exported by the schema module. It defines exactly 24 names, their order, units, integer-valued features, time constants, eligible protocols and special compatibility rules. The machine-readable data schema documents this contract as version 2.0.0. Fig. 3.5 shows the calculation groups. Appendix I gives the complete ordered schema, and Table 3.2 summarizes its interpretation.

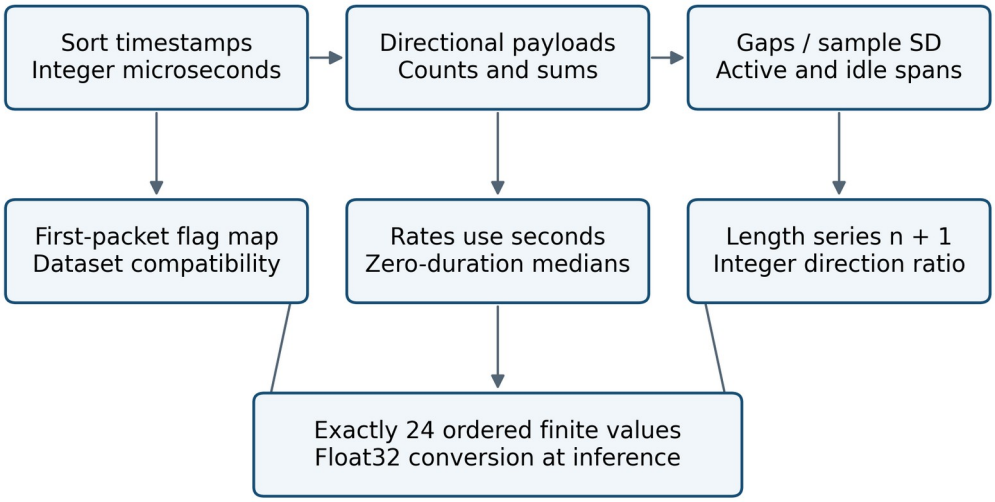


Fig. 3.5 Feature-engineering workflow under the current 24-feature contract. Timing features use microseconds; rates use seconds.

Table 3.2 Feature families and current semantics

Family	Features / role	Critical convention
Size / direction	Counts, directional payload totals and means	Direction follows earliest sorted packet for features.
Timing	Duration, flow and directional IAT, active/idle means	Microseconds; sample standard deviation.
Rates	Flow Bytes/s; Flow Packets/s	Per second; median fills when duration is zero.
Length shape	Packet Length Mean and Std	First payload occurs twice in n + 1 series.
Flags	Six named Flag Count columns	Binary first-packet mapping; not packet totals.
Balance	Down/Up Ratio	Floor(backward / forward).

3.7.1 Timing and directional measurements

Packets are stably sorted by integer microseconds derived from their timestamps. Feature direction is anchored to the earliest packet after sorting. This usually agrees with the flow builder's first-arrival orientation but can differ when delivery order and timestamp order differ. The distinction is retained as an implementation limitation instead of being hidden behind the general phrase "first packet". Empty packet sequences produce zeros, although the integrated classifier excludes flows with fewer than two packets.

For sorted timestamps t_1 through t_n expressed in microseconds, duration is $D = \max(0, t_n - t_1)$. Inter-arrival gaps are $g_i = t_i - t_{i-1}$. Flow IAT Mean averages all gaps, and forward/backward IAT means average gaps within their respective directions. IAT denotes inter-arrival time. Flow IAT Std uses the sample standard deviation with denominator $m - 1$ for m gaps, returning zero for fewer than two gaps. These units and denominators matter when reproducing thresholds in a trained model.

3.7.2 Payload lengths, rates and direction ratio

For each packet, the extractor uses payload_length when available and falls back to observed length otherwise. Directional totals sum payload bytes, and directional means divide by the relevant packet count. The overall Packet Length Mean and Packet Length Std use the series consisting of the first payload followed by all payloads. The first value is therefore counted twice, producing $n + 1$ observations. This is intentional local compatibility behavior and must not be rewritten as the ordinary mean of n packet lengths.

For nonzero duration, Flow Bytes/s = total payload bytes $\times 10^6 / D$ and Flow Packets/s = $n \times 10^6 / D$. The displayed flow duration remains in seconds. If D is zero, the extractor supplies the recorded training rate-fill constants: 3715.0378579999997 payload bytes/s and 69.742244285 packets/s. These values preserve the documented preprocessing convention; they are not measurements of an instantaneous physical rate.

Down/Up Ratio is floor(n_b / n_f), with zero when no forward packets exist. It is not forward divided by backward, and it is not a fractional ratio. For example, three backward packets and two forward packets yield 1.0. This correction is important because the supplied guide describes the older inverse convention. The removal of Average Packet Size from the candidate schema is described as near-duplication, not exact algebraic equivalence under the current $n + 1$ length semantics.

3.7.3 First-packet flags and activity periods

The six feature columns named Flag Count are binary indicators of the first sorted TCP packet. The contract maps FIN Flag Count to R, SYN Flag Count to P, RST Flag

Count to C, PSH Flag Count to S, ACK Flag Count to A, and URG Flag Count to F. The names thus preserve dataset-compatible column semantics rather than conventional names for accumulated TCP flags. Non-TCP packets contribute zero to these binary indicators. Any future explanation must use this mapping when describing what was actually observed.

Active and idle periods are derived from gaps greater than five million microseconds. Such a gap closes a preceding positive active span and is recorded as idle. The final active span is not added. If the flow was closed by FIN, the final closing packet is excluded from activity-state updates. A short uninterrupted flow can consequently have Active Mean equal to zero even though communication occurred. The feature is a statistic of recorded segmented periods, not the total amount of time the host was working.

3.7.4 Input validation and remaining semantic risks

FeatureVector enforces the vector length and provides mapping, tuple and list views. Integer-schema values are converted through integer coercion and then represented as floats. The model boundary verifies that all supplied values are finite. A raw sequence has no semantic names attached beyond its position, so order correctness remains the caller's responsibility. The from_mapping helper fills missing names with zero; this convenience is not a substitute for verifying completeness in an external ingestion service.

The current code repairs the guide's seconds-versus-microseconds mismatch. Feature names, units and order agree across the contract and model metadata. However, this establishes a local contract rather than universal equivalence to every historical CICFlowMeter output. Timing resolution, retransmissions, padding, packet loss, FIN behavior and mid-flow starts can still alter the representation. Independent packet-level reference captures remain necessary to validate those boundary cases.

3.8 Module 5: XGBoost inference

MLDetectionEngine loads the saved JSON booster and feature/class metadata before processing traffic. It verifies exact feature order and exact class order. The class

sequence is BENIGN, Brute Force, DDoS, DoS and Port Scan, corresponding to identifiers zero through four. The loader prefers models/ and retains support for an older xgboost/ artifact directory. A dedicated import helper avoids confusing that local directory with the installed Python library.

Fig. 3.6 shows the inference path. Values are converted to a one-row float32 array, wrapped in an XGBoost DMatrix with feature names, and passed to the booster. The best_iteration attribute is 520, so inference uses the half-open iteration range (0, 521). The audited model contains 521 boosting rounds. The highest of five probabilities supplies the predicted class and confidence. No scaler, one-hot encoder or principal-component transformation is loaded by the runtime detector.

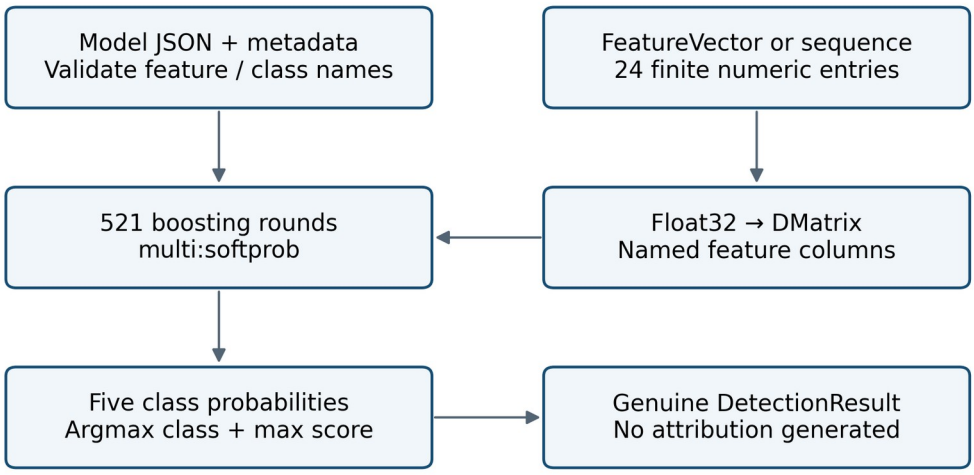


Fig. 3.6 XGBoost loading and inference. Confidence is the maximum model probability and is not a calibrated probability of a real intrusion.

The predict_many method loops over rows and invokes single-row prediction; it is not a vectorized batch API. A convenience predict function constructs a new engine per call, whereas the integrated pipeline reuses one engine. The distinction affects runtime cost. Model-load and feature-validation errors are explicit, while try_predict can suppress unexpected prediction failures after validation. A failed inference is therefore not equivalent to a BENIGN prediction and should be counted separately in any operational monitor.

3.9 Notebook training and evaluation workflow

The notebook contains a substantive training workflow, contrary to the older guide's claim that training code is absent. It loads eight local CIC CSV files, strips column whitespace, removes duplicates, handles invalid rates, maps labels and selects 24 features. The final selection merges FTP-Patator and SSH-Patator into Brute Force, combines four DoS subclasses, renames PortScan and excludes categories outside the five-class task. The notebook retains historical outputs from inconsistent states, so source code and saved outputs are identified separately.

The split cell specifies an 80% training partition and a 20% temporary partition, then splits the latter equally into validation and test sets. Both operations stratify by class and use random_state=42. Recorded sizes are 2,014,489 training rows, 251,811 validation rows and 251,812 test rows. Fig. 3.7 depicts this recorded methodology and the separate audit replay. The diagram does not claim a fresh end-to-end retraining run.

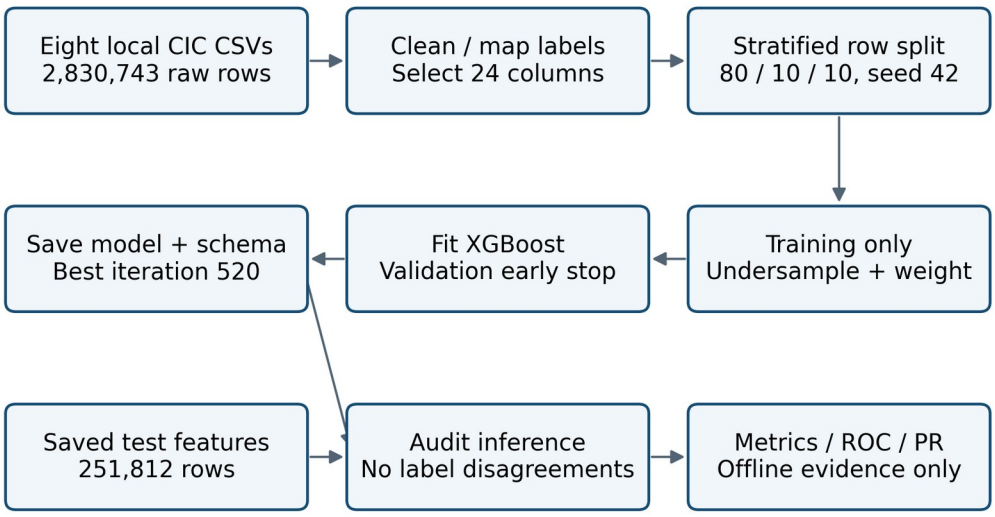


Fig. 3.7 Recorded training/evaluation methodology and audit replay. Early notebook outputs are inconsistent; the saved test artifacts and regenerated inference are independently checked.

Only the training partition is subsequently undersampled. BENIGN is reduced to 150,000 rows, all recorded attack rows are retained, and the combined training population becomes 487,388. Balanced class weights are computed on this population and passed as sample weights. The recorded Brute Force weight is approximately

13.313, much larger than the BENIGN weight of approximately 0.650. This provides a plausible mechanism for favouring rare-class recall, although the causal effect on false positives cannot be isolated without an ablation.

Table 3.3 records the final training-cell parameters. Early stopping monitors validation_1 multiclass log loss with a patience of 100 rounds and save_best=True. The metadata records XGBoost 3.4.1, while requirements.txt constrains runtime XGBoost to versions below 3.0. The audit therefore explicitly records its installed version, 2.1.4, and verifies the resulting predictions rather than assuming identical floating-point probabilities across versions.

Table 3.3 Recorded final notebook training configuration

Parameter	Value	Source / qualification
Objective / classes	multi:softprob / 5	Notebook cell 56; five-class artifacts
Training rows after sampling	487,388	Cell 55 output; BENIGN target 150,000
Candidate estimators / rate	3,000 / 0.05	Cell 56 source
max_depth / min_child_weight	8 / 2	Cell 56 source
subsample / colsample_bytree	0.85 / 0.85	Cell 56 source
gamma / reg_alpha / reg_lambda	0.05 / 0.05 / 1.5	Cell 56 source
Tree method / device	hist / cpu	Cell 56 source
Random seed / jobs	42 / −1	Recorded training configuration
Early stopping	100 rounds; validation_1 mlogloss	save_best=True
Best round / validation loss	520 / 0.02570706015121968	Booster attrs and metadata agree
Model library metadata	XGBoost 3.4.1	Recorded, not the audit runtime
Audit library	XGBoost 2.1.4	All saved labels reproduced

3.10 Pipeline orchestration, completion and output

IntrusionDetectionPipeline accepts raw packets, parsed metadata, batches or live capture. It owns the builder, feature engine and detector, exposes counters, and supports reset, flush and close. Completed flows must contain at least two packets and

use TCP or UDP to be classified. A pipeline detection combines the completed flow, feature vector and genuine detector result. The genuine result is appended to internal results before the optional display transformation is applied to the callback copy.

The CLI can print class, confidence, endpoint key, observed packet and byte counts, duration and all five probabilities. Its --detections-log option appends one JSON object per displayed result, including genuine prediction fields and a demo_fallback indicator. The serialization does not include the complete feature vector, even though the in-memory PipelineResult retains it. Existing JSON Lines (JSONL) files therefore support prediction analysis but cannot fully reconstruct the feature evidence for an explanation service.

The pipeline catches feature-extraction or detection exceptions and continues. Such failures are debug-logged but do not have a dedicated failed-flow counter. The accepted-packet counter increments before checking whether usable addressing exists, and can therefore include metadata later rejected by the flow builder. These details limit the interpretation of counters as throughput or success metrics. A future operational version should distinguish parsed, flow-eligible, classified, intentionally skipped and failed observations.

3.11 Traffic profiles, parameter mapping and generation

The profile JSON stores empirical reference statistics and a source-population count of 2,518,112. profile_loader reads medians and discriminative-feature blocks, and parameter_mapper converts those values into bounded connection, delay and payload settings. The profile numbers are reference statistics, not the actual observed 24-feature vector. Fig. 3.8 shows the separation between the profile, traffic stimulus, capture and detector.

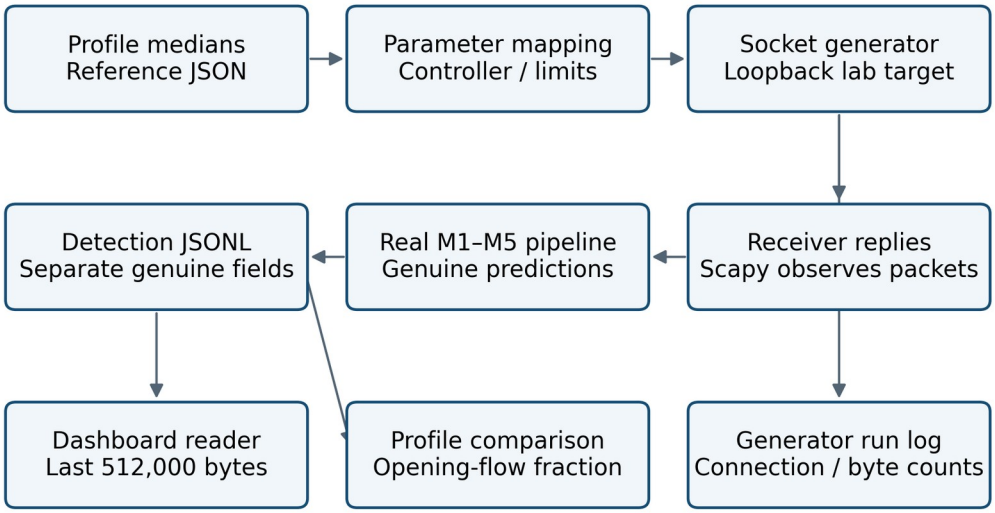


Fig. 3.8 Generator/receiver interaction and evidence paths. Profile reference medians do not bypass capture or enter the detector as synthetic feature vectors.

Five generator classes implement distinct laboratory shapes. BenignTrafficGenerator performs ordinary request/response exchanges. PortScanTrafficGenerator uses TCP connect probes over a bounded range; a field named send_syn_only does not turn it into a raw SYN-only packet generator. BruteForceTrafficGenerator uses a line-oriented laboratory session with six request/deny exchanges. DDoSTrafficGenerator requests a small laboratory path with bounded concurrency, while DoSTrafficGenerator sends browser-like requests in bursts separated by pauses.

The configured page response is 11,600 bytes; the DoS request builder targets 330 bytes and the DDoS request is a short Hypertext Transfer Protocol (HTTP) request. These shape choices are implementation details used to approximate selected training-flow patterns. They should not be interpreted as genuine exploitation or as proof that the response semantics match every member of a dataset attack class. The sender's packets_sent counter records application send operations and is not a measured count of packets on the wire.

The controller validates a private/local target, port and duration, manages a stop event and worker thread, records progress, and writes a structured generator run log. Target checks reject public destinations and support a documented private/loopback policy. The configuration declares caps of 300 seconds, 20 connections/s, 50 packets/s, ten concurrent connections, fifty ports and 512 payload bytes. These are laboratory

controls; not every constant is a packet-level enforcement mechanism, and receiver page bytes are distinct from the generator’s payload parameter.

3.12 Receiver services and validation modes

LabEchoServer binds a laboratory endpoint and handles connections in worker threads. It returns a fixed page for the configured page paths and responds to laboratory authentication-like lines with a denial line. receiver_server exposes this service as a CLI for a separate receiver. Connectivity helpers probe endpoint availability and offer platform-specific capture guidance. A TCP service must be available for request/response profiles, whereas a closed-port response can be meaningful to a port-probe profile.

LocalValidationRunner can start the actual pipeline on loopback and collect in-memory results. It converts them into ValidationRecord objects containing target profile, prediction, probabilities and all observed feature values. Its stop method closes the pipeline and joins the worker with a timeout, but the pipeline’s locally created capture engine is not exposed for an explicit stop request. This is a lifecycle concern: closing the classifier path does not itself demonstrate that a background sniff loop has terminated cleanly.

The launcher’s validation utility runs profiles sequentially, records time windows and compares genuine predictions for connection-opening flows. Its opening-flow criterion is an initial flag string exactly equal to S. The denominator therefore excludes closing-handshake tails and other flows that do not satisfy the criterion. This restricted match fraction is useful for a controlled demonstration, but is not the same statistic as end-to-end accuracy over all classified flows or recall against independently verified malicious events.

3.13 Windows launcher and process lifecycle

start_mlcn.bat selects an available project Python executable and invokes mlcn_launcher. The launcher checks platform, Npcap availability, administrative requirements, interface discovery, model readiness and relevant ports. It identifies the local adapter and the Npcap loopback device, starts the receiver service and pipeline,

and manages the Streamlit child process. Child environments configure UTF-8 output, the single-PC mode, the session file and demonstration fallback behavior.

The process wrapper gives each child an independent process group where supported. Shutdown attempts an interrupt, then termination and finally kill if required. The launcher's default LaunchConfig enables demo_fallback, uses lab port 8080 and a fifty-port span, and sets inactivity to five seconds. A --no-demo-fallback launcher option is available. Reproducible scientific validation must record this option or equivalent environment setting, rather than relying on a launcher default intended for presentation.

3.14 Streamlit console and demonstration provenance

The current Streamlit application is a two-panel operations console. The left panel controls loopback generation at 127.0.0.1. The right panel reads receiver JSONL records and presents a final session result or latest flow, probabilities, flow metadata and recent history. The reader tails at most 512,000 bytes, tolerates incomplete JSON lines and filters by recording time. This is implemented receiver visibility, correcting the older guide's claim that Streamlit has only generator-side status.

Demonstration fallback creates a synthetic five-class probability distribution when the genuine class differs from the selected profile. Its synthetic selected-class confidence is sampled between 0.91 and 0.975. The genuine prediction remains available separately. Session verdicts apply a 90% opening-flow match rule and also check the aggregate class. Synthetic distributions represent presentation behavior and cannot be included in accuracy, calibration or explanation-quality measurements.

The source currently names the fallback result "Fetching RESULT" and its disclaimer "Interpolated Values", while older log records use "FABRICATED DEMO RESULT". The dashboard's fabricated property recognizes an "Interpolated Values" source or a per-flow demo_fallback boolean. A session record using "Fetching RESULT" lacks that boolean, creating a risk of mislabelling its displayed source. The six observed test failures include expectations affected by this terminology and

presentation drift. The report therefore uses the explicit term synthetic demonstration output whenever such values are discussed.

3.15 Test architecture and integration evidence

The tests cover packet normalization, module handoffs, canonical keys, timeouts, feature calculations, schema validation, real saved-model inference, pipeline lifecycle, laboratory generators, launcher behavior, dashboard parsing and fallback logic. Some tests patch sniff or construct packets directly. tests/lab_traffic_simulation.py explicitly describes its packet sequences as approximations of Windows loopback behavior. Successful tests using those sequences establish consistency under the simulated inputs, not successful capture from a physical interface.

The audit executes the complete unittest suite. An initial sandboxed run encounters three socket-binding errors in addition to six assertion failures. A rerun with loopback sockets permitted removes the environment errors and produces 196 passing tests and six failures out of 202. The tests that load the real classifier and exercise the simulated profile chain pass in that run. Chapter 4 interprets the failures and keeps this evidence separate from saved Windows-oriented validation records.

3.16 Proposed Gemini XAI architecture

The proposed workflow is genuine prediction plus associated evidence, followed by a structured Gemini request, validated explanation generation and a human-readable report. Fig. 3.9 identifies every new element as proposed. It operates after classification so that a language-model timeout cannot delay or replace the underlying detector. No network call to Gemini is made during this investigation, and no application implementation is added.

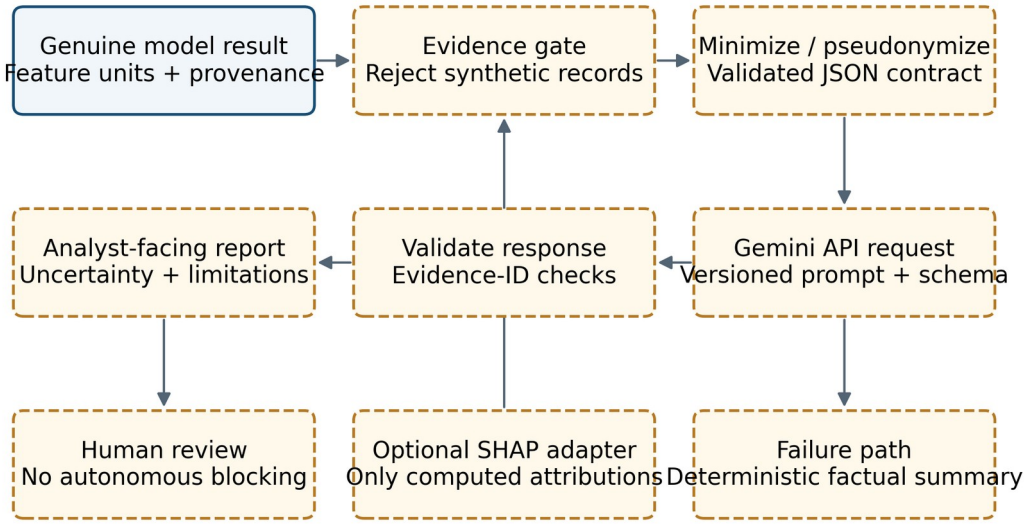


Fig. 3.9 Proposed Gemini explanation architecture. Dashed blocks are unimplemented design elements; only genuine evidence is eligible for explanation.

3.16.1 Input contract and grounding

A proposed EvidenceEnvelope contains an evidence identifier, capture timestamp, flow identifier, model hash, feature-schema version, genuine class, all five genuine probabilities, feature values with units, capture-quality flags and provenance status. The classification component supplies these fields from the in-memory result or an expanded future evidence sink. The current JSONL export is insufficient for complete feature grounding because it omits the feature vector. Appendix V provides the proposed contract as an interface specification rather than as a recorded API payload.

The envelope explicitly rejects a synthetic displayed distribution as model evidence. If a demo_fallback record is encountered, the service may use separately verified genuine fields only when their provenance is complete; otherwise it returns an insufficient-evidence summary. Feature values are validated for finiteness, order, units and schema version. The flag mapping is included in explanatory metadata so that a column name cannot be interpreted as a conventional count without checking its definition.

Each explanatory observation cites an evidence identifier and a field path. For example, a statement about Flow Duration cites its microsecond value and any displayed conversion to seconds. The system does not claim that a feature caused the

decision merely because its value is large. Optional SHAP contributions would arrive through a separate attribution object with its method, model version, background/reference definition and output space. If absent, the report states that feature attribution was not computed.

3.16.2 Request and response design

The proposed request separates trusted instructions from untrusted evidence values. A versioned instruction specifies the task, permitted evidence, class vocabulary, uncertainty language and JSON response schema. Evidence is serialized as data rather than concatenated into free-form instructions. A model identifier is configured and logged at deployment time; the report avoids selecting an unverified future model name. Structured output is used for parseability, with semantic validation performed locally after generation.

The response contains a factual summary, evidence-linked observations, limitations, alternative explanations, recommended human checks and a provenance block. It repeats the genuine prediction without changing its label or probabilities. It avoids unsupported statements about attacker identity, successful compromise, credential theft, attack campaign scale or business impact. A recommendation such as reviewing an associated service log is distinguishable from an assertion that the service was compromised.

3.16.3 Privacy, uncertainty and reliability

The proposed privacy boundary excludes raw payloads, credentials and unnecessary host identifiers. Where endpoint continuity is useful, pseudonymous identifiers can preserve within-session relationships without disclosing actual addresses. Retention, provider data-handling terms and authorization for cloud processing must be selected for the deployment environment. These are proposed controls, not verified properties of an existing Gemini integration.

Reliability requires timeout limits, bounded retries, an input-size limit and an idempotent evidence identifier. A cache key includes model, schema and prompt versions to prevent reuse of an explanation for different evidence. Invalid JSON,

unknown evidence references, altered probabilities or ungrounded factual claims cause the response to be rejected. The fallback is a deterministic template showing the verified class, scores and limitations. The design does not allow the language model to issue firewall changes or launch diagnostic traffic autonomously.

3.16.4 Explanation acceptance criteria

A proposed evaluation set should include correct and incorrect detections, low-confidence cases, missing features, corrupted records, synthetic fallback records and prompt-like text embedded in untrusted fields. Acceptance measures include factual consistency with the envelope, evidence-reference coverage, numerical fidelity, privacy leakage, refusal on insufficient evidence, and analyst usefulness. Latency and API cost are measured separately from capture-to-classification latency. No accuracy, usefulness score or runtime estimate is assigned before those experiments exist.

Chapter 4
Results and Discussions

4.1 Evidence populations and experimental setup

Four populations must be distinguished. The raw local CSV collection contains 2,830,743 flow rows. The profile artifact describes a selected five-class population of 2,518,112 rows. The saved offline test table contains 251,812 rows. The saved launcher validation file contains 1,159 flow records from a loopback-oriented laboratory run. These populations have different origins, filtering rules and labels. Their metrics are not pooled into a single accuracy figure.

The audit runs under Python 3.9.6 on macOS 26.7.1 with an ARM64 platform, XGBoost 2.1.4 and scikit-learn 1.6.1. It loads the existing model without training or modifying it. The 24 saved test columns are converted to float32 and passed to the booster using rounds zero through 520. The original evaluation summary and notebook record XGBoost-related metadata separately. No Windows/Npcap capture is newly performed on this macOS host.

Table 4.1 records the source and interpretation of each result family. All saved test features are finite. Recomputed predictions agree with every saved class label, and the confusion matrix reconstructed from the saved labels exactly matches the provided matrix. The probability scores differ slightly between the audit runtime and the stored winning-class confidences, so historical summary metrics and regenerated score-based results are labelled distinctly.

Table 4.1 Result provenance and permitted interpretation

Result family	Source artifact	Evidence level
Offline hard-label metrics	models/xgboost_test_predictions.csv	Recomputed from 251,812 saved labels
Model replay	xgboost_ids_multiclass.json + same test features	Recomputed: zero class disagreements
Historical probability metrics	models/xgboost_evaluation_results.json	Recorded original summary

Result family	Source artifact	Evidence level
ROC / PR / AP	workspace/regenerated_probabilities.npz	Regenerated scores, XGBoost 2.1.4
Training process	Untitled-1.ipynb, cells 54–57	Recorded source/outputs; stale earlier states
Laboratory validation	logs/launcher/validation_report.txt and validation_detections.jsonl	Saved run; no fresh Npcap capture
Test suite	workspace/test_run_unrestricted.txt	Executed audit: 202 tests, six failures

4.2 Class distribution and imbalance

Fig. 4.1 shows the verified test distribution on a logarithmic count axis, with exact counts and class percentages. BENIGN accounts for approximately 83.25% of the test population, whereas Brute Force accounts for approximately 0.36%. This disparity is large enough that overall accuracy can remain high even when the rare class generates substantial false positives. The logarithmic axis is used only to make all class counts visible; it does not alter the values used for metrics.

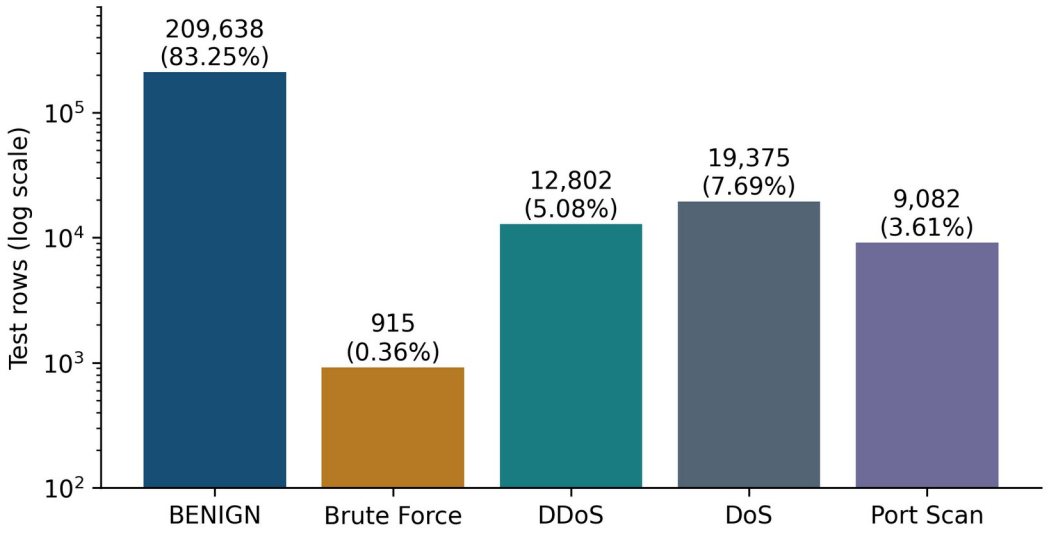


Fig. 4.1 Verified class distribution in models/xgboost_test_predictions.csv, n = 251,812. Bar heights use a logarithmic count axis.

The test set preserves the original selected-population proportions rather than the undersampled training proportions. This is appropriate for measuring performance on the recorded evaluation distribution, but it also exposes the consequences of the training reweighting strategy. A model that gives the rare class more training influence

can achieve high recall while assigning too many benign examples to that class. The present data support describing that trade-off; they do not establish which particular hyperparameter caused it.

4.3 Overall offline classification results

Table 4.2 reports the verified summary metrics. Accuracy is 0.9942377647, equivalent to 99.4238%, with 250,361 correct predictions and 1,451 errors. Balanced accuracy is 0.9955679604, equivalent to the mean of the five class recalls. Macro F1 is 0.9335848515, while weighted F1 is 0.9947219616. The lower macro score reflects the influence of the Brute Force class when each class receives equal weight.

Table 4.2 Offline evaluation summary

Metric	Value	Provenance
Accuracy	0.9942377647	Saved summary; hard-label metrics cross-checked
Balanced accuracy	0.9955679604	Saved summary; hard-label metrics cross-checked
Macro precision	0.8993072360	Saved summary; hard-label metrics cross-checked
Macro recall	0.9955679604	Saved summary; hard-label metrics cross-checked
Macro F1	0.9335848515	Saved summary; hard-label metrics cross-checked
Weighted precision	0.9957345549	Saved summary; hard-label metrics cross-checked
Weighted recall	0.9942377647	Saved summary; hard-label metrics cross-checked
Weighted F1	0.9947219616	Saved summary; hard-label metrics cross-checked
Cohen kappa	0.9808581605	Saved summary; hard-label metrics cross-checked
Matthews correlation coefficient	0.9809986349	Saved summary; hard-label metrics cross-checked
Log loss (saved)	0.0244827153	Recorded original probability metric
ROC-AUC OVR macro (saved)	0.9999122875	Recorded original probability metric
ROC-AUC OVR weighted (saved)	0.9998860870	Recorded original probability metric

Precision for a class is TP/(TP + FP), recall is TP/(TP + FN), and F1 is twice the product of precision and recall divided by their sum. TP, FP and FN denote true positives, false positives and false negatives respectively. Accuracy weights every row equally, balanced accuracy weights each class recall equally, and weighted averages use class support. These definitions explain why high balanced accuracy can coexist with lower macro precision: balanced accuracy does not directly penalize the false-positive burden within a rare predicted class.

The saved multiclass log loss is 0.0244827153 and the saved macro one-versus-rest ROC-AUC is 0.9999122875. The regenerated values are 0.0244703876 and 0.9999122997 respectively. The maximum absolute difference between regenerated winning confidence and the saved Confidence column is approximately 0.00059742. This small probability drift does not change any winning class, but it is sufficient to justify identifying the runtime and avoiding a claim of bitwise reproduction.

4.4 Per-class results

Table 4.3 presents precision, recall, F1 and support for every class. Fig. 4.2 provides a visual comparison of those same values. DDoS, DoS and Port Scan have high offline precision and recall in this saved population. Brute Force is the clear exception in precision and F1, despite its recall being close to 99%. The table supports a class-specific conclusion rather than a uniformly strong statement about all attack categories.

Table 4.3 Verified per-class offline evaluation

Class	Precision	Recall	F1	Support
BENIGN	0.999472	0.993618	0.996536	209,638
Brute Force	0.531378	0.990164	0.691603	915
DDoS	0.996419	0.999688	0.998050	12,802
DoS	0.979628	0.995252	0.987378	19,375
Port Scan	0.989639	0.999119	0.994356	9,082

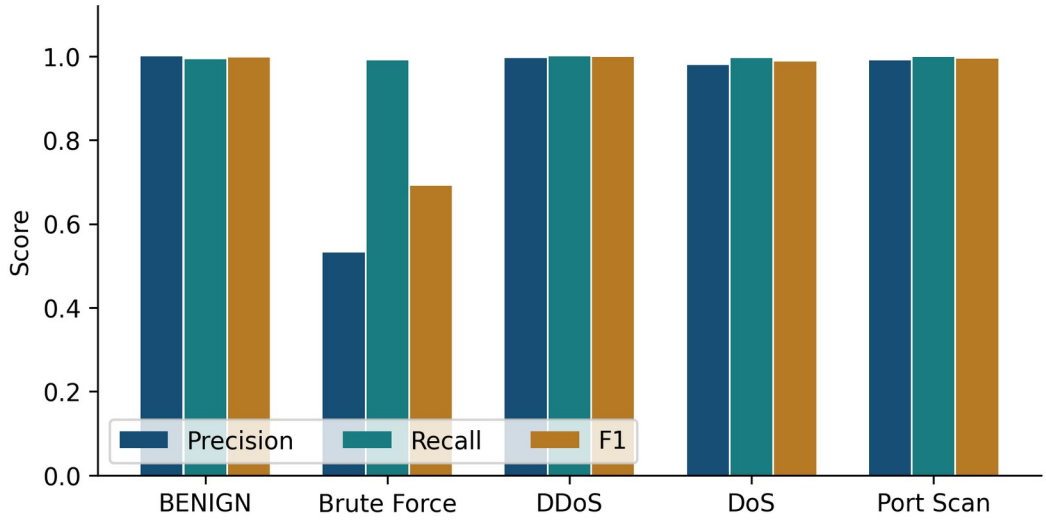


Fig. 4.2 Per-class precision, recall and F1 reconstructed from saved offline labels. Brute Force precision is substantially below the other classes.

A support of 915 gives less evidence about the range of Brute Force behaviors than the 209,638 BENIGN rows give about the recorded benign population. Moreover, individual rows may not be statistically independent because multiple flows can arise from the same host, service or scripted activity. A simple binomial confidence interval would not address that dependence. The report therefore avoids presenting a narrow interval as a guarantee of cross-session or cross-network performance.

4.5 Confusion matrix and error analysis

Fig. 4.3 visualizes the verified confusion matrix, and Table 4.4 supplies its exact integer values in editable form. Rows are actual labels and columns are predicted labels. Colour is logarithmic to expose small error cells alongside the large benign diagonal. Matrix orientation is stated explicitly because transposing its interpretation changes the error analysis.

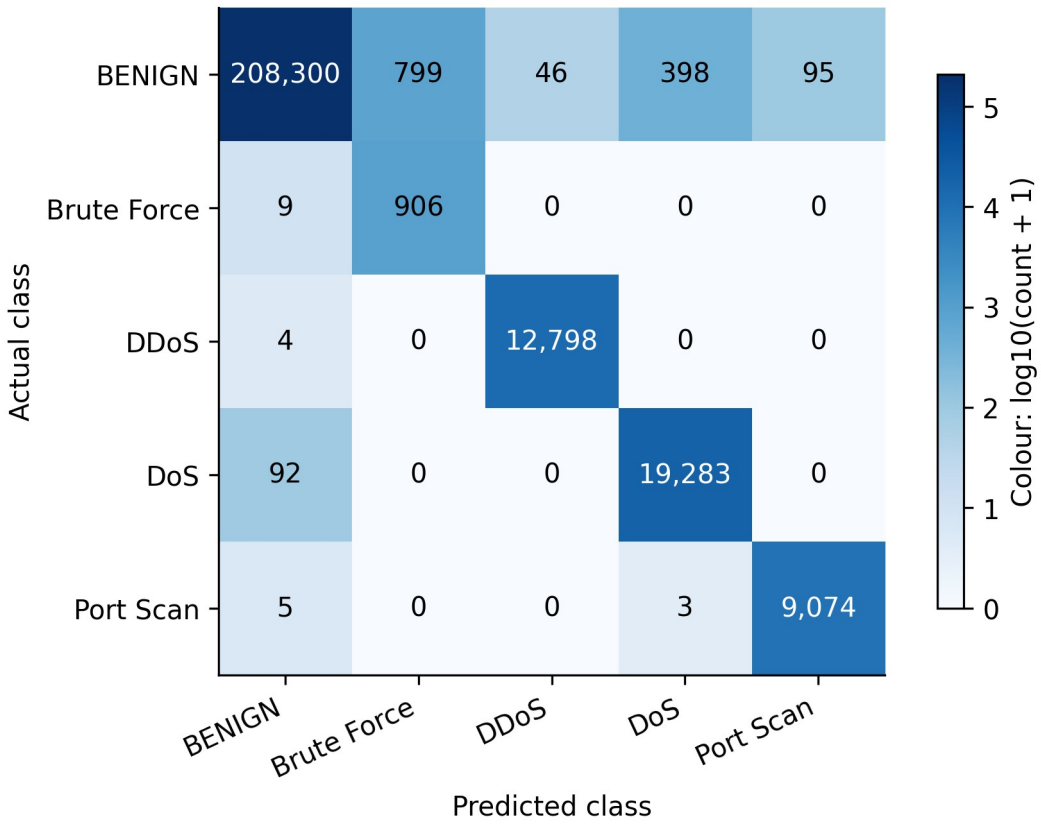


Fig. 4.3 Verified offline confusion matrix. Cell annotations are exact counts; colour represents log10(count + 1). Rows are actual and columns predicted.

Table 4.4 Exact confusion matrix (actual rows, predicted columns)

Actual / predicted	BENIGN	Brute Force	DDoS	DoS	Port Scan
BENIGN	208300	799	46	398	95
Brute Force	9	906	0	0	0
DDoS	4	0	12798	0	0
DoS	92	0	0	19283	0
Port Scan	5	0	0	3	9074

Brute Force has 906 true positives, nine false negatives and 799 false positives. All 799 false positives originate from BENIGN rows. Consequently, 906/(906 + 799) = 53.1378% precision, and 46.8622% of Brute Force predictions in this test set are incorrect. The guide's count of 1,338 false positives combines the entire benign error row across several predicted classes. That value is the number of BENIGN rows assigned to any attack label, not the Brute Force false-positive count.

The 1,338 BENIGN errors divide into 799 Brute Force, 46 DDoS, 398 DoS and 95 Port Scan predictions. They account for approximately 92.21% of all 1,451 errors. Other classes mainly lose a small number of rows to BENIGN: nine Brute Force, four DDoS, ninety-two DoS and five Port Scan rows. Three additional Port Scan rows are predicted as DoS. These patterns show that the principal offline error burden is benign traffic being classified as attacks.

The mean saved winning confidence is approximately 0.98847 for correct predictions and 0.72097 for incorrect predictions. This association suggests that score-aware review could be useful, but it does not validate a threshold policy. Thresholds selected using this test set would contaminate its role as final evaluation data. Any change should be selected using validation data and then tested once on a separate untouched population.

4.6 ROC and precision–recall analysis

The saved CSV includes the winning confidence but not all five class probabilities. ROC and PR curves cannot be reconstructed correctly from that single score alone. The audit therefore regenerates the complete probability matrix from the saved model and all 24 test features, verifies label agreement, and stores the matrix with its runtime provenance. Fig. 4.4 and Fig. 4.5 are based on those regenerated scores.

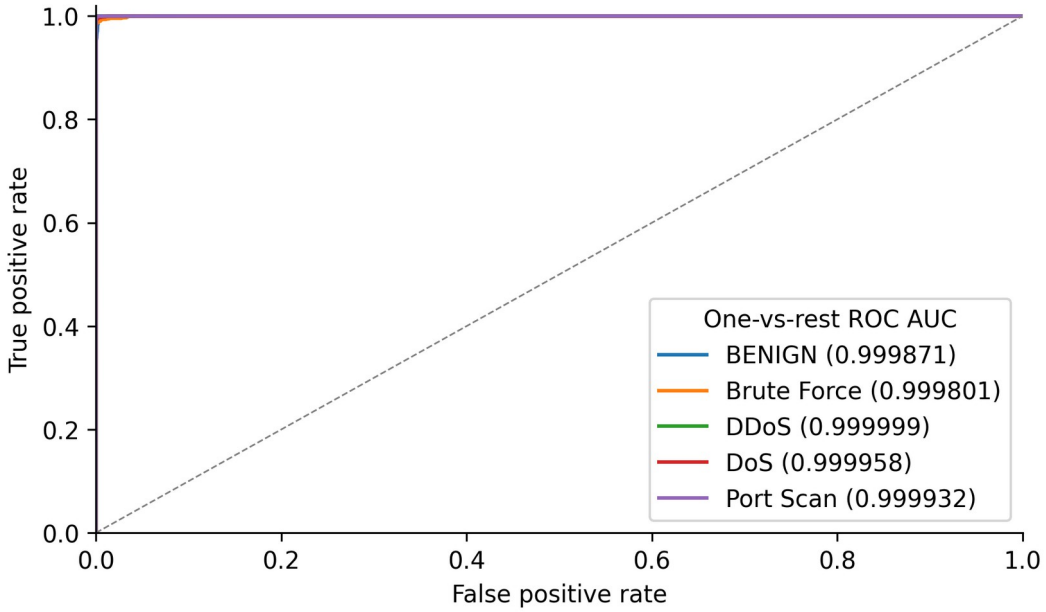


Fig. 4.4 One-versus-rest ROC curves regenerated from the saved booster and test features using XGBoost 2.1.4. These are offline ranking results.

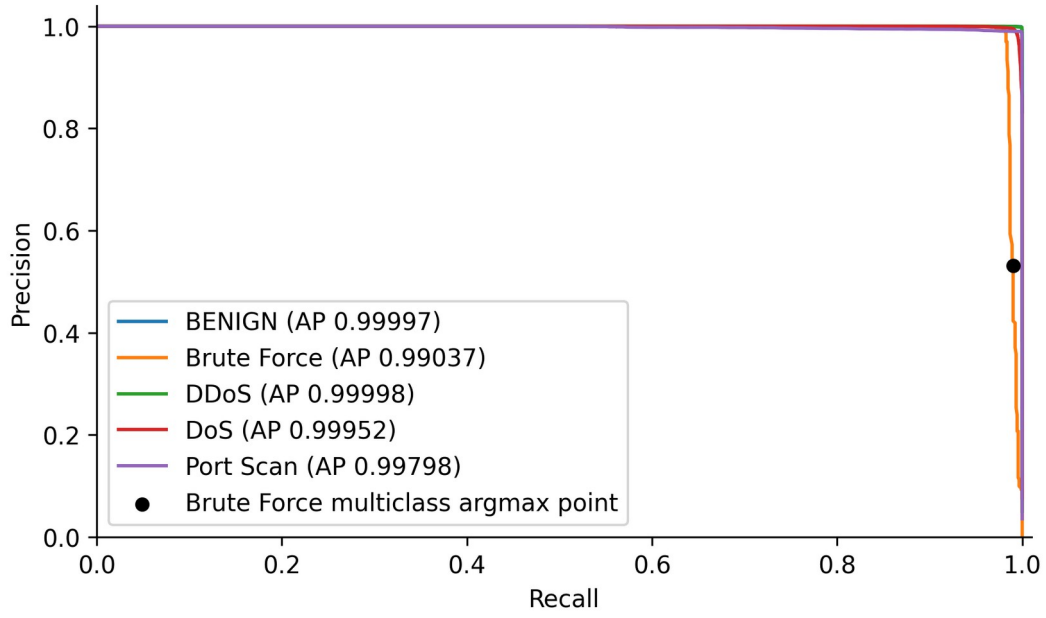


Fig. 4.5 One-versus-rest PR curves from regenerated probabilities. The black point shows the separate Brute Force multiclass argmax operating point.

Table 4.5 reports per-class ROC-AUC and average precision (AP). AP is calculated from the precision–recall ranking and is not the same as precision at the multiclass argmax decision. Brute Force can have high AP and still have low argmax precision because a thresholded one-versus-rest ranking answers a different question from choosing the greatest score among five classes. The black point in the PR plot makes

this difference explicit rather than suggesting that the curve removes the observed false positives.

Table 4.5 Regenerated one-versus-rest ranking metrics

Class	ROC-AUC	Average precision
BENIGN	0.999870968	0.999973756
Brute Force	0.999801135	0.990365026
DDoS	0.999998976	0.999980677
DoS	0.999958076	0.999518537
Port Scan	0.999932343	0.997980152

These curves describe performance on the saved test population. They do not establish calibrated probabilities, resistance to evasion, performance on newly collected hosts or detection of unseen classes. The close agreement between regenerated and saved macro AUC is an artifact-consistency result. It does not resolve upstream questions about how the original split and preprocessing interacted with dataset dependence.

4.7 Performance comparisons and baseline interpretation

A valid local comparison is an always-BENIGN baseline evaluated on the same saved test labels. It obtains approximately 83.2518% accuracy and 20% balanced accuracy, because it recalls the benign class and misses all four attack classes. Table 4.6 contrasts this simple baseline with XGBoost. The comparison establishes that the model extracts information beyond the majority-class frequency on this evaluation set.

Table 4.6 Same-test-population baseline comparison

Method	Accuracy	Balanced accuracy	Macro F1
Always BENIGN	0.832518	0.200000	0.181721
Saved XGBoost	0.994238	0.995568	0.933585

No matched final test artifacts for Random Forest, support-vector classification or neural networks are verified in the repository. Introductory notebook prose mentioning several algorithms is not evidence of an executed final comparison. Similarly, published scores from different datasets cannot serve as a controlled baseline. A future comparison should use the same cleaned rows, split indices, feature contract,

hyperparameter-selection budget and operating-point criteria for every candidate method.

4.8 Training provenance and notebook consistency

The raw local CSV row counts match the eight-file count table recorded early in the notebook. The final split and balancing outputs are consistent with the profile artifact's selected population and saved test supports. However, earlier cleaning and selection cells retain contradictory outputs. One removal output starts again from 2,830,743 rows and reports remaining missing/infinite values; a later selection output contains only BENIGN and DDoS and gives 2,401,124 rows. Those states cannot be the immediate clean five-class input to the recorded final split.

This inconsistency is evidence of non-linear or partially stale notebook execution. It does not invalidate the independently checked saved test predictions, but it prevents certification of the notebook as a clean top-to-bottom reproduction of the original training run. The report accordingly describes training hyperparameters and recorded split counts as notebook evidence, while describing metric replay as independently executed evidence. No new model training is performed during the reporting task.

Preprocessing is written before the split, including median filling of invalid rate values. That design can expose information from future evaluation rows to a transformation estimated over the complete population. The potential effect is not measured here. The row-wise stratified split also does not enforce separation by source day, host or session. A stronger protocol would persist split membership and fit every learned preprocessing operation only on training data.

4.9 Resolution of the historical feature mismatch

The current feature engine computes timing in microseconds and rates per second. Contract tests check duration and inter-arrival units, payload accounting and schema order. The older guide's warning that live seconds are sent directly into a microsecond-trained model is therefore not accurate for the audited code. This is a supported correction to the implementation account, not a claim that every source of live-to-training distribution shift has disappeared.

Remaining risks include the inferred historical flag semantics, first-packet duplication in length statistics, padding behavior, first-observed direction, FIN segmentation and differences among capture platforms. The pipeline applies a 120-second maximum age but retains a configurable inactivity rule. Retransmissions and partial captures can change packet counts and lengths even if units are correct. The packet simulations include a port-scan retransmission sensitivity test, reinforcing that small traffic-shape changes can affect predicted labels.

4.10 Saved end-to-end laboratory validation

The saved validation report records five sequential profile runs. Its total flows sum to 1,159, matching validation_detections.jsonl. Every record in that file identifies REAL ML and has demo_fallback false. The genuine class counts over all records are BENIGN 826, Brute Force 4, DDoS 190, DoS 24 and Port Scan 115. Those aggregate counts include closing tails and are not themselves profile-specific correctness counts.

To corroborate the report, the JSONL file is partitioned into consecutive blocks using the recorded totals 595, 9, 382, 50 and 123. Their time spans are sequential and separated. Selecting first_packet_flags equal to S within those blocks reproduces the reported opening-flow counts. Table 4.7 gives exact reconstructed matches, and Fig. 4.6 compares their counts. The original ProfileWindow objects are not persisted; this reconciliation is therefore explicitly a reconstruction guided by the saved report, not an independent recovery of ground-truth windows.

Table 4.7 Saved laboratory validation, opening-flow denominator

Profile	All flows	Opening	Matches	Match rate
BENIGN	595	298	298	100.0000%
Brute Force	9	5	4	80.0000%
DDoS	382	190	188	98.9474%
DoS	50	24	23	95.8333%
Port Scan	123	119	115	96.6387%

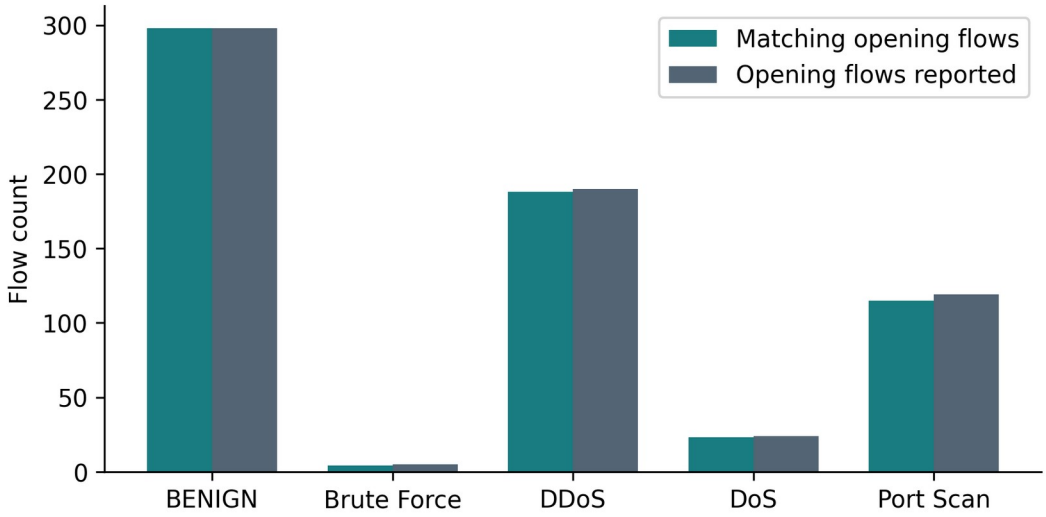


Fig. 4.6 Saved laboratory opening-flow counts, reconciled using consecutive JSONL blocks and the saved report's profile totals. Matching is relative to the selected profile.

The Brute Force run contains four matching opening flows out of five, giving 80%, below the utility's 90% rule. The saved report labels its final demonstration verdict fabricated. The underlying genuine detections remain valid observations; a fabricated final verdict cannot turn the failed profile criterion into a successful experiment. The other opening-flow matches are 298/298 BENIGN, 188/190 DDoS, 23/24 DoS and 115/119 Port Scan. These results demonstrate selected laboratory profile recognition under the recorded setup, with very limited Brute Force sample size.

A separate detections.jsonl contains 1,554 records, including eight session records and 219 per-flow fallback records. Its source strings include both historical fabricated labels and the current fetching label. It is deliberately excluded from pooled performance calculation. Records in that file may still preserve genuine fields, but display values must not be treated as classifier outputs. No packet-capture file is supplied to independently reconstruct the original live features or measure capture losses.

4.11 Executed tests and failure interpretation

The unrestricted loopback-capable audit run executes 202 tests in 28.078 seconds. It has 196 passes, six failures and no errors. The failing cases are listed in Table 4.8. Several compare the expected explicit fabricated-result string with the current "Fetching RESULT" label. Dashboard-related failures additionally expose inconsistent

recognition or presentation of fabricated verdicts. They are not reported as detector accuracy failures, but they directly affect trust in displayed results.

Table 4.8 Failures in the 202-test audit run

Test module / class	Failing case	Interpretation
test_dashboard / TestReceiverData	test_verdict_takes_priority_and_flags_fabricated	Session fabricated-source recognition
test_dashboard / TestStreamlitApp	test_launch_invokes_generator_and_shows_fabricated_result	Displayed fabricated-state expectation
test_demo_fallback / TestSessionResult	test_printed_fabricated_result_shows_selected_and_disclaimer	Printed label/disclaimer expectation
test_live_validation / TestLauncherReport	test_exit_code_requires_every_profile_real	Report wording expectation
test_live_validation / TestSummarize	test_attributes_by_flow_start_and_counts_only_opening_flows	Expected explicit fabricated verdict label
test_live_validation / TestSummarize	test_no_capture_is_fabricated_not_real	Expected explicit fabricated verdict label

Passing tests establish useful local properties: feature-contract behavior, saved-model loading, representative class vectors, module handoffs and simulated packet-profile classification. They do not prove Windows capture privileges, Npcap driver operation or long-running stability on the target machine. The test named run_live wiring uses a mocked capture path. The report preserves these distinctions so that a green test for orchestration is not represented as a fresh real-time capture experiment.

4.12 Runtime observations and implementation costs

A single audit batch prediction for 251,812 rows takes 3.1602 seconds with the booster configured for four threads. Model loading and DMatrix construction occur before the timed interval. This is an offline inference observation on one host and one run. It is not a packet throughput benchmark, a confidence interval, or an estimate of the latency experienced by an individual live flow.

Live latency includes flow completion, feature computation, single-row DMatrix creation, prediction, callback processing and display refresh. Idle flows may wait for another packet or a flush before expiration is acted upon. The builder scans active flows during expiration, and completed results retain packet metadata. These implementation costs motivate future throughput, memory and packet-loss

measurements. No unsupported packets-per-second or production-capacity claim is derived from the batch timing.

4.13 Threats to validity and supported interpretation

Internal validity is limited by stale notebook outputs, preprocessing before splitting, inferred compatibility semantics and absent original live packet captures. External validity is limited by the selected five classes, a historical dataset, controlled loopback traffic and no independently labelled deployment population. Construct validity is limited when a selected profile is used as the expected class: matching that label measures conformity to a laboratory stimulus, not necessarily detection of malicious intent.

The supported interpretation is that the repository implements a connected packet-to-flow classifier, reproduces the saved offline labels, and contains recorded laboratory evidence of several profile matches. It also retains substantial Brute Force precision limitations and presentation-source defects. Those qualifications are integral to the result. High aggregate offline metrics support continued engineering investigation; they do not establish a production-ready autonomous security system.

Chapter 5
Summary and Conclusions

5.1 Summary of the investigation

The investigation traces the MLCN implementation from capture through classification and assesses its evidence at each boundary. The current source contains five integrated processing modules, a canonical 24-feature contract, a saved XGBoost multiclass model, local CIC flow data, a training notebook, laboratory generators, receiver services, launcher utilities and a receiver-aware Streamlit console. The report also documents the optional display fallback that can substitute synthetic values for presentation while retaining the genuine model result.

Independent replay confirms all 251,812 saved class predictions and the exact saved confusion matrix. Original diagrams document the architecture, while charts report verified class counts, scores and regenerated probability rankings. Existing live-validation records are reconciled with their summary using an explicitly stated method. The institutional document structure is preserved, with unknown student, guide and outcome-mapping details left for authorized completion.

5.2 Objectives achieved

Table 5.1 summarizes achievement against the investigation objectives. The distinction between completed engineering work and incomplete deployment validation is maintained. A proposed Gemini explanation workflow is specified in enough detail to guide implementation, but it is not included among implemented or empirically validated contributions.

Table 5.1 Objective achievement and remaining work

Objective	Outcome	Remaining requirement
Trace integration	Completed source and interface audit	Long-running deployment verification
Verify feature contract	Current microsecond contract documented	Independent reference-capture parity

Objective	Outcome	Remaining requirement
Reproduce offline results	All saved labels reproduced	Clean retraining from persisted splits
Assess laboratory results	Saved genuine records reconciled	New labelled runs and capture files
Audit provenance	Synthetic branch and six failures documented	Repair source labels and enforce genuine-only evaluation
Specify Gemini XAI	Detailed proposed contracts and controls	Implementation and measured explanation evaluation

5.3 Conclusions derived from the results

First, a usable classifier integration depends on an exact measurement contract. Current MLCN timing features use microseconds, rates use seconds, and payload and flag rules are explicitly defined. This repairs the historical timing mismatch described in the supplied guide. The unusual compatibility semantics remain important limitations when transferring to another extractor or retraining on corrected data.

Second, the saved offline model exhibits strong overall classification performance on its recorded test population: 99.4238% accuracy and 99.5568% balanced accuracy. These findings are independently reproduced from saved labels and verified by model replay. The aggregate result must be read alongside macro F1 of 0.93358 and Brute Force precision of 0.53138. The latter corresponds to 799 false positives among 1,705 Brute Force predictions.

Third, the available laboratory evidence supports selected profile recognition but has a narrower interpretation than general intrusion-detection accuracy. Brute Force matches only four of five opening flows in the saved validation run. Synthetic final verdicts and transformed display probabilities provide no evidence of improved detection. The test failures around their labelling show why provenance must be a visible, machine-checkable property of every reported result.

Fourth, the notebook is informative but not a fully certified reproducibility pipeline. It records splits, balancing, hyperparameters and training output, yet its earlier saved states are inconsistent. A repeatable training package must preserve preprocessing

decisions, dataset identities, split membership and environment versions. The report's successful inference replay addresses consistency of saved artifacts, not complete recreation of training history.

5.4 Contributions and limitations

The principal project contribution is a modular laboratory platform linking captured traffic to schema-validated multiclass inference. Its supporting tools allow bounded traffic generation, receiver observation and investigation of how flow shape affects classification. The present report adds a verified account of those mechanisms, corrected metric interpretation, reproducible result extraction and an explanation design that keeps factual evidence separate from generated language.

The main limitations are restricted class coverage, incomplete live provenance, weak rare-class precision, unresolved test failures in presentation behavior, possible flow-state concurrency and shutdown issues, and missing long-duration performance measurements. There is no implemented attribution engine, Gemini reporting service or incident database. There is also no controlled comparative benchmark establishing that XGBoost outperforms alternative models under an identical protocol.

5.5 Specific future scope

The first priority is trustworthy evidence handling. Demonstration fallback should be disabled in evaluation, its source should be unambiguous in every record type, and the six failing tests should be resolved through an explicit product decision. A new evidence sink should store feature values, schema/model hashes, capture context and completion reason alongside genuine predictions. These changes would make later error analysis and explanation grounding auditable.

The second priority is repeatable data preparation and evaluation. A clean training script should replace state-dependent notebook execution, fit imputation on training data, preserve source/session identifiers and create group-aware held-out sets. Raw-data hashes and extractor versions should be stored with every model. The Brute Force decision boundary should then be studied through additional representative benign and

attack flows, validation-only threshold selection, calibration and comparisons with matched baselines.

The third priority is live-system measurement. Repeated authorized laboratory captures should vary operating system, interface, segmentation, retransmissions, background traffic and flow termination. Measurements should include packet loss, eligible-flow coverage, failure counts, memory growth and completion-to-prediction latency. An explicit stop channel, synchronized shared state and a timed expiration mechanism should be evaluated before extending the system to longer-running monitoring.

The final priority is the proposed explanation layer. A validated evidence adapter should precede any Gemini integration, with privacy minimization, versioned prompts and deterministic fallback behavior. SHAP may be evaluated as a separate feature-attribution component. Human assessment should compare factual correctness and usefulness against a simple template before claiming that generated narrative improves incident triage.

Chapter 6
Appendix

6.1 Appendix I — Complete feature schema

Table 6.1 gives the ordered contract used by the runtime model. The order is part of the interface and must be preserved when exporting or constructing a vector. Types describe the numerical meaning; the inference array itself is float32. The formula column summarizes actual implementation behavior, including the exceptional flag mapping and length series. It is not a claim that every network-flow tool uses the same definitions.

Table 6.1 Complete ordered 24-feature runtime contract

No. / feature	Unit / type	Implemented calculation
1. Flow Duration	microseconds / int	last_ts - first_ts
2. Total Fwd Packets	packets / int	packets in first-packet direction
3. Total Backward Packets	packets / int	packets in opposite direction
4. Fwd Packets Length Total	payload bytes / int	sum fwd payload
5. Bwd Packets Length Total	payload bytes / int	sum bwd payload
6. Fwd Packet Length Mean	payload bytes / float	fwd payload / fwd packets
7. Bwd Packet Length Mean	payload bytes / float	bwd payload / bwd packets
8. Flow Bytes/s	payload bytes per second / float	payload / (duration_us / 1e6)
9. Flow Packets/s	packets per second / float	packets / (duration_us / 1e6)
10. Flow IAT Mean	microseconds / float	mean gap between packets
11. Flow IAT Std	microseconds / float	sample std of gaps
12. Fwd IAT Mean	microseconds / float	mean gap between fwd packets
13. Bwd IAT Mean	microseconds / float	mean gap between bwd packets
14. Packet Length Mean	payload bytes / float	mean of [first] + all payloads
15. Packet Length Std	payload bytes / float	sample std of [first] + all payloads
16. FIN Flag Count	binary / int	first packet has RST
17. SYN Flag Count	binary / int	first packet has PSH
18. RST Flag Count	binary / int	first packet has CWR
19. PSH Flag Count	binary / int	first packet has SYN

No. / feature	Unit / type	Implemented calculation
20. ACK Flag Count	binary / int	first packet has ACK
21. URG Flag Count	binary / int	first packet has FIN
22. Down/Up Ratio	ratio / float	floor(bwd packets / fwd packets), 0 if no fwd
23. Active Mean	microseconds / float	mean active period closed by idle gap
24. Idle Mean	microseconds / float	mean gap > 5 s

For an illustrative arithmetic check, consider three timestamps at 0, 1,000 and 3,000 microseconds with payloads 10, 20 and 30 bytes, and directions forward, backward and forward. Duration is 3,000 microseconds and gaps are 1,000 and 2,000 microseconds. Their mean is 1,500 and sample standard deviation is approximately 707.107 microseconds. Flow Bytes/s is 20,000 and Flow Packets/s is 1,000. The overall length series is 10, 10, 20, 30, with mean 17.5 bytes. These are illustrative formula calculations, not captured experimental results.

The same example gives two forward packets, one backward packet and Down/Up Ratio equal to zero because floor(1/2) is zero. This result illustrates why interpreting that feature as a continuous proportion would be incorrect. A zero-duration case would instead use the contract's fixed rate fills. A future independent extractor should be checked against examples of this kind and against real reference captures before it is permitted to supply model inputs.

6.2 Appendix II — Implementation inventory

Table 6.2 lists the audited implementation files by responsibility, including supporting packages beyond the five core modules. Entry-point and initializer files are small but important because they determine how command-line and package execution reach the same implementation. The full file-level Secure Hash Algorithm 256-bit (SHA-256) inventory is supplied in the report workspace. Virtual environments and version-control internals are excluded from scientific source claims.

Table 6.2 Audited implementation and artifact inventory

Path	Files / interfaces	Responsibility
packet_capture/	__init__.py, __main__.py, capture.py, cli.py, formatter.py, interfaces.py, models.py, parser.py	Capture engine, interface discovery, CLI display and formatting; __main__ entry point.
packet_parsing/	__init__.py, models.py, parser.py, validation.py	Scapy interpretation, immutable metadata and field normalization.
flow_builder/	__init__.py, builder.py, models.py	Canonical keys, mutable active accumulators and immutable completed flows.
feature_engineering/	__init__.py, contract.py, engine.py, models.py, schema.py	Canonical contract, schema exports, feature algorithms and vector model.
ml_detection/	__init__.py, engine.py, models.py, schema.py, xgboost_import.py	Artifact paths, model loading, feature validation, class results and import compatibility.
pipeline/	__init__.py, __main__.py, cli.py, engine.py, models.py	End-to-end orchestration, result serialization, CLI logging and entry point.
traffic_generator/	__init__.py, __main__.py, config.py, connectivity.py, controller.py, dashboard_data.py, lab_server.py, parameter_mapper.py, profile_loader.py, receiver_server.py, safety.py, streamlit_app.py, traffic_generator.py, traffic_profile.py, validation.py	Profiles, configuration, mapping, bounded socket generators, receiver, controller, safety, connectivity, validation and dashboard.
mlcn_launcher/	__init__.py, __main__.py, launcher.py, preflight.py, processes.py, validation.py	Windows/Npcap preflight, child processes, automatic validation and launcher CLI.
demo_fallback/	__init__.py, fallback.py, session.py	Session state, synthetic display transformation and genuine-result preservation.
data/	03_finalize_common_feature_schema.py, __init__.py, common_feature_schema.py	Common schema generation, validation, JSON and narrative schema artifacts.
models/	Booster JSON; feature metadata; test predictions; classification report; confusion matrix; evaluation JSON	Trained and evaluation artifacts.
tests/	Fifteen test_*.py files, fixtures.py, lab_traffic_simulation.py and __init__.py	202 test cases executed during audit.
Top level	README.md; PROJECT_IMPLEMENTATION_GUIDE.md; Untitled-1.ipynb; requirements.txt; start_mlcn.bat; .gitignore	Documentation, recorded training, dependencies and Windows entry point.
logs/	Launcher JSONL, validation summary, session JSON, Streamlit log and 17 generator run JSONs	Historical execution records; mixed real/synthetic sources separated.

The tests directory contains fixtures, a packet-profile simulation utility and fifteen test modules. Its test categories include common-schema generation, feature contracts, packet parsing, flow construction, detection, pipeline integration, traffic generation,

local validation, launcher behavior, dashboard behavior and demonstration provenance. Assets contain four project images and the compact traffic-profile JSON. The images are contextual design artifacts; the JSON supplies reference statistics used by the generator. The top-level query file contains a short Npcap lookup token and has no verified executable role.

6.3 Appendix III — Configuration and operating modes

Table 6.3 distinguishes standalone builder settings from integrated pipeline and launcher settings. This avoids mixing values from different entry points. Laboratory port filtering affects capture and callback display; eligibility still requires TCP or UDP and at least two packets. A source record should include the selected configuration whenever results from different runs are compared.

Table 6.3 Configuration by execution context

Setting	Standalone builder	Integrated / launcher
Idle timeout	60 s	Pipeline 60 s; launcher 5 s; LocalValidationRunner 10 s
Maximum duration	300 s	Pipeline 120 s
FIN termination	Disabled	Enabled for integrated pipeline
Active-flow capacity	100,000	100,000 pipeline default
Eligible protocols	Builder accepts keyed metadata	Classifier TCP / UDP only
Minimum flow packets	No builder classification rule	Two packets
Activity threshold	Not a builder feature	5 s in feature engine
Lab port / span	Not applicable	Launcher 8080 / 50 ports
Streamlit port	Not applicable	8501
Demo fallback	Not applicable	Launcher enabled by default; --no-demo-fallback disables
Generator duration / connection cap	Not applicable	300 s / 20 connections per second
Generator concurrency / scan span	Not applicable	10 connections / 50 ports

The requirements file specifies Scapy at least 2.5 and below 3, XGBoost at least 2 and below 3, NumPy at least 1.20 and below 3, and Streamlit at least 1.37 and below 2.

The training metadata's XGBoost 3.4.1 lies outside that declared runtime range. The audit successfully loads the model with 2.1.4 and verifies its predictions, but a future release should pin and test a consistent development and deployment environment rather than leave this discrepancy implicit.

6.4 Appendix IV — Reproducibility instructions

The report workspace contains the audit script, source inventory, metric results, probability matrix, plotting script, manuscript and document-generation script. Reproduction of the report's metric checks begins from the unmodified model and saved prediction CSV. The audit computes labels and metrics without fitting a new model. Its hash inventory provides a way to determine whether source artifacts have changed since the report was generated.

Table 6.4 gives concise commands and their intended roles. Commands for capture are instructions for an authorized laboratory environment and are not a record of execution during this audit. Interface names are platform-specific and must be selected from actual discovery output. The Windows launcher is intentionally platform-specific; the macOS audit host cannot certify Npcap operation by running ordinary unit tests.

Table 6.4 Reproducibility and laboratory command reference

Command	Role / qualification
python -m unittest discover -s tests -t . -v	Run existing tests. The audit has six failures; do not claim a fully passing suite.
python reports/workspace/audit.py	Recompute evidence under the report environment; no training.
python -m pipeline -l	Discover exact capture-interface names.
python -m pipeline -i <interface> --lab-port 8080 --detections-log <report-workspace-log>	Capture on the authorized laboratory interface; omit demo fallback and set its environment toggle to 0.
start_mlcn.bat --no-demo-fallback	Windows launcher with genuine-only display configuration.
start_mlcn.bat --validate --no-demo-fallback	Windows laboratory profile validation; preserve genuine log and run context.
python -m traffic_generator.receiver_server --help	Inspect receiver CLI options before a separate-machine laboratory run.

A complete future training reproduction should first repair notebook state dependencies, replace machine-specific paths with parameters, persist exact source ordering and split membership, and pin library versions. It should then reconstruct the selected class population, fit preprocessing only on the training partition, perform the recorded undersampling and weighting, and train with validation-only early stopping. New results should be stored under a new model identity rather than overwrite the current evidence silently.

The report-generation environment is independent from the application environment. It adds document and plotting libraries solely under reports/workspace. The original application modules and model artifacts are not edited. Test execution uses temporary test directories where the suite requests them and disables bytecode generation for the audit run. Any reproduction should retain failed-test output as well as successful checks, because presentation failures affect the interpretation of results.

6.5 Appendix V — Proposed explanation contract

Table 6.5 defines the proposed explanation envelope and response. Names in this table are design suggestions, not an implemented public API. The required source of classification fields is the genuine detector output. The actual JSONL format must be extended or bypassed through an in-memory adapter to include all feature evidence. A missing feature block cannot be reconstructed from probabilities alone.

Table 6.5 Proposed Gemini evidence and response contract

Object / field	Required meaning	Validation rule
EvidenceEnvelope.identity	evidence_id, model_hash, schema_version, capture time	Required; stable versioned identity
EvidenceEnvelope.prediction	Genuine class and five probabilities	Class order, finite scores, sum tolerance
EvidenceEnvelope.features	All 24 names, values, units and semantic notes	Exact schema, no missing units
EvidenceEnvelope.provenance	Capture context, quality flags, genuine source	Reject synthetic display evidence
EvidenceEnvelope.attribution	Optional method-specific feature contributions	Absent means no attribution claim
Request.prompt_version	Trusted task and response schema	Separate from untrusted evidence content

Object / field	Required meaning	Validation rule
Response.observations	Factual statements with evidence IDs and field paths	All references resolve; numbers agree
Response.uncertainty	Missing evidence and alternative interpretations	No unsupported compromise/identity claim
Response.recommendations	Human review steps	No autonomous blocking or generated commands executed
Response.provenance	Input ID and model/prompt versions	Exact link to validated request

The response validator should verify that every evidence reference resolves, all repeated numerical values agree within an explicit tolerance, and no generated output changes the original prediction. Explanatory language should distinguish an observed statistic from an attribution and distinguish both from a causal claim about an incident. A synthetic record, invalid schema, absent provenance or privacy-policy failure should produce a factual refusal or deterministic summary rather than a plausible narrative.

6.6 Appendix VI — Supplementary evidence and traceability

Table 6.6 records the local raw CSV file sizes in rows, as counted during this audit. The filenames are retained to allow exact correspondence with the workspace. Their sum is 2,830,743. The source population in the profile artifact is smaller because it describes a selected and cleaned task population. The notebook’s inconsistent intermediate outputs prevent treating that entire reduction as a freshly certified preprocessing lineage.

Table 6.6 Local CIC CSV row counts

CSV filename	Rows
Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv	225,745
Friday-WorkingHours-Afternoon-PortScan.pcap_ISCX.csv	286,467
Friday-WorkingHours-Morning.pcap_ISCX.csv	191,033
Monday-WorkingHours.pcap_ISCX.csv	529,918
Thursday-WorkingHours-Afternoon-Infilteration.pcap_ISCX.csv	288,602
Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv	170,366
Tuesday-WorkingHours.pcap_ISCX.csv	445,909
Wednesday-workingHours.pcap_ISCX.csv	692,703

Table 6.7 provides selected artifact hashes for traceability. Full hashes and the complete inventory remain available in the machine-readable workspace file. Model, schema and evaluation artifacts should travel together; changing one without a new identity can invalidate a seemingly reproducible prediction. The audit date identifies the local inspection, while the institutional academic-year label remains the supplied template's 2025–26.

Table 6.7 Selected artifact SHA-256 hashes

Artifact	SHA-256
models/xgboost_ids_multiclass.json	7e0b2cf18a647c36e9feac4050662472 3a83cf7abc4f743eb7f33923cacb60a5
models/xgboost_ids_features.json	45b1c9654981cb0862ab21141708e927 be297333eec76b13df87c40c596e2a68
models/xgboost_test_predictions.csv	618a3f22d39d65ad8ab3346d84c70907 cf9db6bd343448c2c51c415cecf8d0e3
models/ xgboost_evaluation_results.json	d7cad27ff607954497ff61084d6bcfae e974967f40ca7351f675004e09c4cb4e
logs/launcher/ validation_detections.jsonl	ea0018e81dc55139c35729174527bda9 e8b1e294aa65f4c3adbaef90d6a4f24f
feature_engineering/contract.py	3e1f4b5ebe00136ee881689a3784d2b2 21206adcd4daee5157754161593cdd06
Untitled-1.ipynb	14ac5fab64715ce8d46569552e509551 7a4fe080272dad90f95bdeecada860e4

Chapter 7

References / Literature Cited

7.1 Bibliography in order of first citation

The following entries are numbered in first-citation order. Online source verification was performed on 21 October 2026. Repository-derived facts cite exact artifact paths in the relevant chapters and appendices; the file inventory records their local provenance. Patent entries identify publications and technical relevance only. They do not assert project novelty or a legal clearance.

[1] Sommer R., Paxson V., “Outside the Closed World: On Using Machine Learning for Network Intrusion Detection,” 2010 IEEE Symposium on Security and Privacy (2010). Available: https://gangw.cs.illinois.edu/class/cs562/papers/Closed_World-IDS-sp10.pdf

[2] Canadian Institute for Cybersecurity, University of New Brunswick, “Intrusion Detection Evaluation Dataset (CIC-IDS2017).” Dataset documentation, including the cited study by Sharafaldin I., Lashkari A. H. and Ghorbani A. A., “Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization,” 4th International Conference on Information Systems Security and Privacy (2018). Available: https://www.unb.ca/cic/datasets/ids-2017.html

[3] Chen T., Guestrin C., “XGBoost: A Scalable Tree Boosting System,” arXiv:1603.02754 (2016), associated conference DOI: 10.1145/2939672.2939785. Available: https://arxiv.org/abs/1603.02754

[4] Handley M., Paxson V., Kreibich C., “Network Intrusion Detection: Evasion, Traffic Normalization, and End-to-End Protocol Semantics,” 10th USENIX Security Symposium, Washington, D.C. (2001). Available: https://www.usenix.org/conference/10th-usenix-security-symposium/network-intrusion-detection-evasion-traffic-normalization

[5] Canadian Institute for Cybersecurity, University of New Brunswick, “DDoS Evaluation Dataset (CIC-DDoS2019).” Dataset documentation and associated study by Sharafaldin I., Lashkari A. H., Hakak S. and Ghorbani A. A., “Developing Realistic Distributed Denial of Service (DDoS) Attack Dataset and Taxonomy,” IEEE 53rd International Carnahan Conference on Security Technology, Chennai (2019). Available: https://www.unb.ca/cic/datasets/ddos-2019.html

[6] Stratosphere Laboratory, “The CTU-13 Dataset: A Labeled Dataset with Botnet, Normal and Background Traffic.” Dataset-provider documentation, accessed 21 October 2026. Available: https://www.stratosphereips.org/datasets-ctu13

[7] Engelen G., Rimmer V., Joosen W., “Troubleshooting an Intrusion Detection Dataset: the CICIDS2017 Case Study,” 2021 IEEE Security and Privacy Workshops, pp. 7–12 (2021), DOI: 10.1109/SPW53761.2021.00009. Available: https://intrusion-detection.distrinet-research.be/WTMC2021/index.html

[8] Arp D., Quiring E., Pendlebury F., Warnecke A., Pierazzi F., Wressnegger C., Cavallaro L., Rieck K., “Dos and Don’ts of Machine Learning in Computer Security,” 31st USENIX Security Symposium, pp. 3971–3988 (2022). Available: https://www.usenix.org/conference/usenixsecurity22/presentation/arp

[9] Saito T., Rehmsmeier M., “The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets,” PLOS ONE, vol. 10, no. 3, e0118432 (2015), DOI: 10.1371/journal.pone.0118432. Available: https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0118432

[10] Lundberg S., Lee S.-I., “A Unified Approach to Interpreting Model Predictions,” arXiv:1705.07874 (2017). Available: https://arxiv.org/abs/1705.07874

[11] Ribeiro M. T., Singh S., Guestrin C., “Why Should I Trust You?: Explaining the Predictions of Any Classifier,” arXiv:1602.04938 (2016). Available: https://arxiv.org/abs/1602.04938

[12] Google, “Structured Outputs,” Gemini API documentation. Accessed 21 October 2026; page update shown as 20 October 2026. Available: https://ai.google.dev/gemini-api/docs/structured-output

[13] Powell M. S., Drozdenko B. M., Roodbeen S. A., “Real-time Network Intrusion Detection System,” US12143406B1, published 12 November 2024; original assignee: United States Department of the Navy. Available: https://patents.google.com/patent/US12143406B1/en

[14] Papamartzivanos D., Bifulco R., Kambourakis G., Gomez Marmol F., “Method and System for Adaptive Network Intrusion Detection,” US20220021695A1, published 20 January 2022; original assignee: NEC Laboratories Europe GmbH. Available: https://patents.google.com/patent/US20220021695A1/en

7.2 Related-patent verification scope

The related-patent review in Section 2.8 verifies two publication records and discusses their relevance to monitored-traffic compatibility and adaptive representations. The checked database is Google Patents, with publication identifiers US12143406B1 and US20220021695A1. Search coverage is intentionally limited to directly relevant accessible records. No exhaustive family search, prosecution-history review or jurisdiction-specific claim analysis is represented as completed.

List of Publications

The repository contains no verified publication by the candidate or project team. Table P.1 records that status in the institutional format. It should be replaced only with an authenticated publication entry, including its actual authors, venue, date and indexing information.

Table P.1 Candidate publications status

Sr. No.	Authors	Title of Paper	Name of International Journals / International Conference	Place and date of Publication with Citation Index
—	Not supplied	No verified publication reported	Not applicable	Not applicable